ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

LLM Benchmark project Report

Professors:

Michele Lombardi

Allegra De Filippo

Students:

Simone Migliaccio (matr.: 0001159303)

Simone Spezia (matr.: 0001167605)

Alberto Varini (matr.: 0001189363)

Contents

1 Introduction

The main goal of this project is to provide a general benchmark to evaluate different LLMs performances under different types of planning domains to better understand in advance what are the planning capabilities of a given LLM. In the following sections the main features of this projects are going to be shown.
Also a quick PDDL starting guide has been created to better gain the necessary knowledge about PDDL this specific language used to define planning problems.

2 General Structure

The root folder contains various sub folders, specifically:

- analysis
- archive
- Benchmark Framework
- results

2.1 analysis

This folder is used to gather all the datas useful for the production fo the report and the slides, contains the summary of the raw scores for the PDDL domains and the jupiter notebooks and the report.

2.2 archive

This folder contains all the material given us by the professor, specifically papers, a list of PDDL planning domains, two github repos given us as a reference and last a Quick PDDL guide written by us to help understand better the language.

2.3 Benchmark Framework structure

The folder Benchmark Framework is the core of all the work we've done. The main goal of this framework was to provide a easy to use and general testing platform to be employed when evaluating the performances of a LLM is needed, The following pages are going to be used to show the folders and the structure of the framework:

```
LLM Benchmark/
├── docs/
├── evaluators/
├── Leonardo script/
├── models/
├── outputs/
├── prompts/
├── protocols/
├── reporting/
├── runner/
├── scripts/
├── slurm logs/
├── tasks/
├── tests/
├── utils/
├── advanced planning evaluation.py
├── clear outputs.py
├── run benchmark.py
└── setup.md
```

docs Contains a single file for the model preparation which i useful because the benchmark can run on a local PC or on an HPC system such as Leonardo. The important operational rule is that GPU benchmark jobs should not download large model weights. Prepare Hugging Face models first, then run benchmarks from local files.

evaluators Provides tools for to understand, validate and measure the capabilities of LLMs to solve planning problems

Leonardo script Contains specific scripts to launch and manage jobs on Leonardo HPC.

models Manages the communication with various LLMs, using "adapters" which creates a common interface for different backends like NVIDIA API.

outputs folder containing all the output from the LLMs, divided in 3 sub folders *raw, parsed, scored* to better separate respectively the raw output from the model,PDDL extracted and strctured plans and validation and metrics results.

prompts contains text files each one used to explain to the LLM the rules of the domain and how to manage the problem in order to solve it.

protocols defines different strategies for test execution, for example direct plan (which is one shot) or iteratoive repair.

reporting contains scripts for creation of teh plots used in the reports.

runner contains scripts useful to launch a singlke test or the whole suite.

scripts contains scripts for the computation of the numerical score assigned to each domain.

slurm logs Used to gather information for debug when the benchmark runs on Leonardo HPC

tasks PDDL problems and related instances selected for the runs, the instances are divided in 3 categories *easy, medium, hard.*

tests Used to run tests in preparation for the real run of the benchmark.

utils contains executable for the validator to be used during the run of benchmark framework

advanced planning evaluation.py it reads benchmark artifacts from the output folder and writes a model-centric JSON report in the folder results.

clear outputs is used to empty a folder during Leonardo tests

run benchmark is the file that runs the whole sequence of tests

The framework and its structure is organized in this way in order to allow teh user to change fundamental parameters such as the numbers of iterations easily and effectively and then run the benchmark again with the modified parameters.

2.4 results

This folder contains all the plots and the output archive of all the analysis process; the outputs folder contains only raw datas the results folder contains the analysis done on the data.

3 PDDL Problems Classification

The first thing to do since we wanted a trustable benchmark and since a lot of PDDL models were already provided as reference material was to classify the problems using a numerical difficulty score. We used a numerical score because it is intuitively understandable by anyone who will ever want to compare the performance of LLMs using these models. Another classification implemented consists into separating problems in different categories based on the type of reasoning they request. To compute the scores the code can be found in the file `score_domains_complexity.py`

3.1 Difficulty Classification

3.1.1 Instances scores

The complexity of a single problem instance is calculated using a weighted logarithmic combination of four key metrics. The use of the logarithm prevents any single feature from dominating the score excessively.

The **Raw Score** is defined as:

$$Raw\_score = 0.45 \cdot \log(1 + A) + 0.20 \cdot \log(1 + G) + 0.20 \cdot \log(1 + N) + 0.15 \cdot T \tag{1}$$

Where:

- **A (Actions):** Number of feasible grounded actions. This is estimated by unifying action parameters with objects that satisfy static preconditions in the initial state.
- **G (Goals):** Total number of atomic conditions in the goal state.
- **N (Numeric Score):** A composite value representing numeric complexity:

  $$N = E_{num} + 2 \cdot P_{num} + 3 \cdot G_{num} + M \tag{2}$$

  where E_{num} are numeric effects, P_{num} numeric preconditions, G_{num} numeric goal conditions, and M is a binary flag for the presence of a metric.
- **T (Topology Score):** Evaluates the spatial/relational complexity based on predicates like *adj* or *connected*:

  $$T = \log(1 + Nodes) + \log(1 + Edges) + Density \tag{3}$$

Obviously a different weight could be assigned to each different part of the computation of raw scores depending on the operator that is doing the tests in order to higlight more for example the number of action present in the instances and so on.
Finally, an **Instance Score (0-100)** is calculated by normalizing the raw score relative to the minimum and maximum scores found within the same domain. The results can be found in the folder `domains_complexity`. Note that also some additional statistics for each domain are computed based on the results obtained from the instances, for example it could be of interest evaluating the mean and the median of the scores obtain form the instances of a single domain.

3.1.2 Domain scores

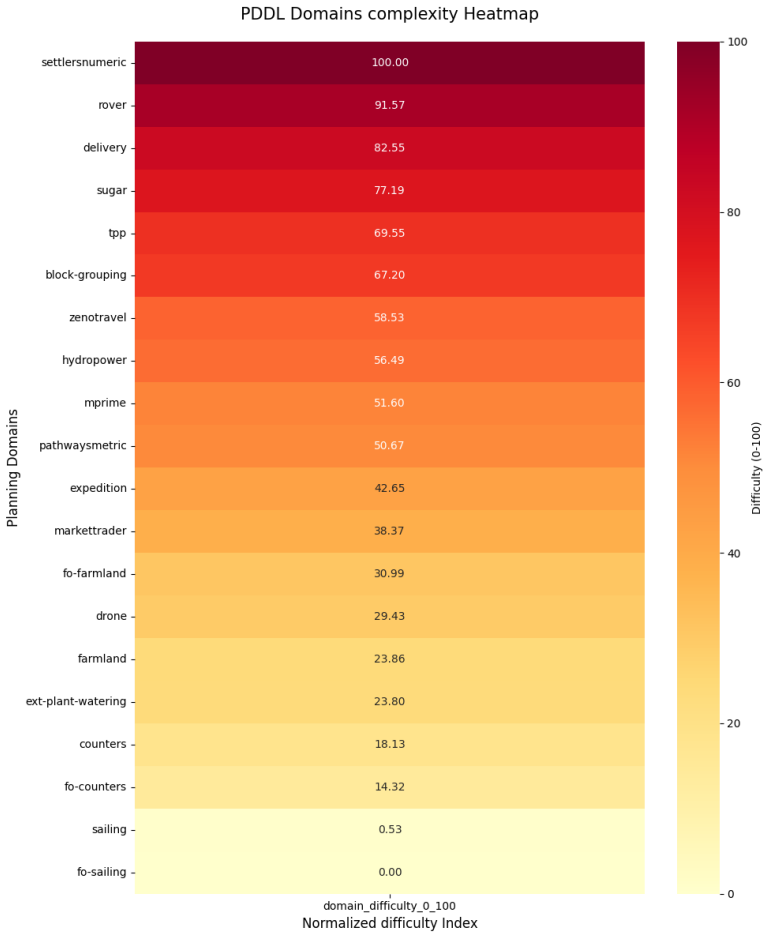
The overall difficulty of a domain is not just the average of its instances, but a reflection of its total state-space and constraint complexity. The **Domain Difficulty (0-100)** is computed by:

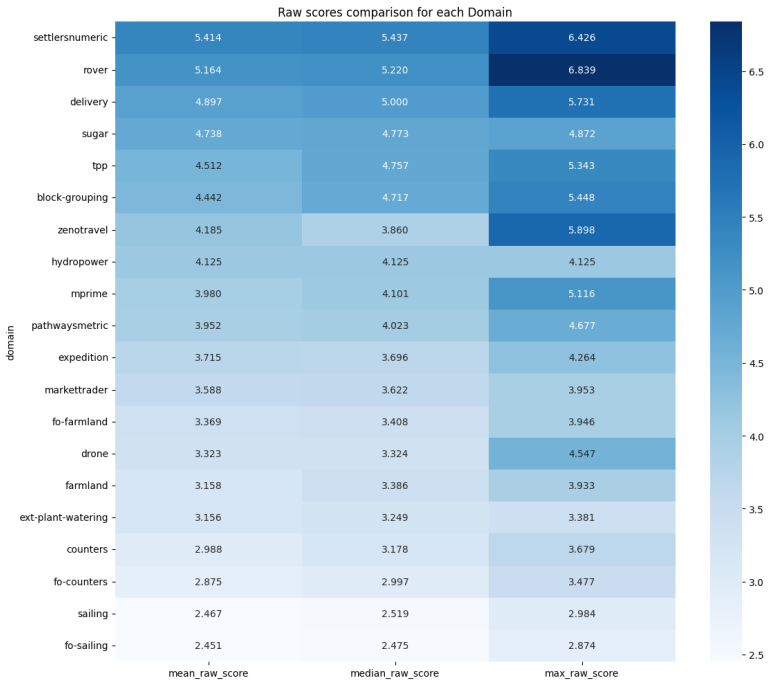
1. Summing the S_{raw} of all instances belonging to the domain to obtain a *Total Raw Score*.
2. Normalizing this total across all tested domains in the benchmark.

This approach ensures that a domain with many complex instances is ranked higher than one with few simple instances, note that obviously the easiest problems gets a domain difficulty equal to 0 (in this case is the "fo-sailing" problem) and the problem with the highest difficulty has domain difficulty equal to 100, (in our case the "settlersnumeric" problem).

3.1.3 Results obtained

In order to obtain a visual representation of the classified problems, we produced heatmaps for both the overall normalized domain scores and the raw scores.





3.2 Reasoning Classification

Here the classification based on the type of reasoning these problems require the LLM to employ is shown. We decided to split the reasoning types into 5 categories:

- **Spatial:** These problems are typically about objects that have to be moved in a 2D (or more dimensional) spaceframe.
- **Numerical Optimization:** These problems require from the LLM an extensive capacity to deal with numerical optimization problems, such as managing complex cost functions and resource constraints.

- **Temporal Planning:** In this kind of problem it is required from the LLM to be able to reach the goal defining an order between objects having timing constraints and relations.
- **Logistic:** In this kind of problems it is required to reach the goal by planning some kind of goods transportation and agents coordination.
- **Logic:** These are abstract puzzles where the state space and actions represent symbolic or mathematical transformations rather than physical interactions. They test the model's ability to reason over purely formal rules without relying on world knowledge.

In the following table the classification of the 20 planning problems is shown, note that in this case all the work have been done by hand and that the problems in the table are ordered form hardest to easiest following the previous difficulty classification. Note also that the evaluation is based on how much the problem belong to the specific domain, for example in the problem "settlersnumeric" there is a dominant Numerical Optimization part hence the problem is going to be classified as a Numerical Optimization problem.

PDDL Problem	**Reasoning Category**
Settlersnumeric	Numerical Optimization
Rover	Logistic
Delivery	Logistic
Sugar	Numerical Optimization
Tpp	Numerical Optimization
Block-Grouping	Spatial
Zenotravel	Logistic
Hydropower	Temporal Planning
Mprime	Logic
Pathwaysmetric	Numerical Optimization
Expedition	Logistic
Markettrader	Numerical Optimization
Fo-farmland	Numerical Optimization
Drone	Spatial
Farmland	Numerical Optimization
Ext-Plant-Watering	Spatial
Counters	Numerical Optimization
Fo-Counters	Numerical Optimization
Sailing	Spatial
Fo-Sailing	Spatial

Table 1: Reasoning Category Classification Tab

3.3 Problems chosen

In order to chose the correct problem to be solved by LLMs we decided to pick 3 sets composed by 3 problems each. The sets are composed by 3 problems with different reasoning categories and specifically from the categories Numericaal Optimization, Spatial and Logistic. Problems related to Temporal Planning and Logic were not chosen since just one problem for each type was present; we suggest, in order to study the capabilities of LLMs in these domains, to use a PDDL problenms dataset specifically containing these kind of problems.
In order to better understand how we picked the problems the following tables are shown, the elements are ordered with decreasing difficulty.

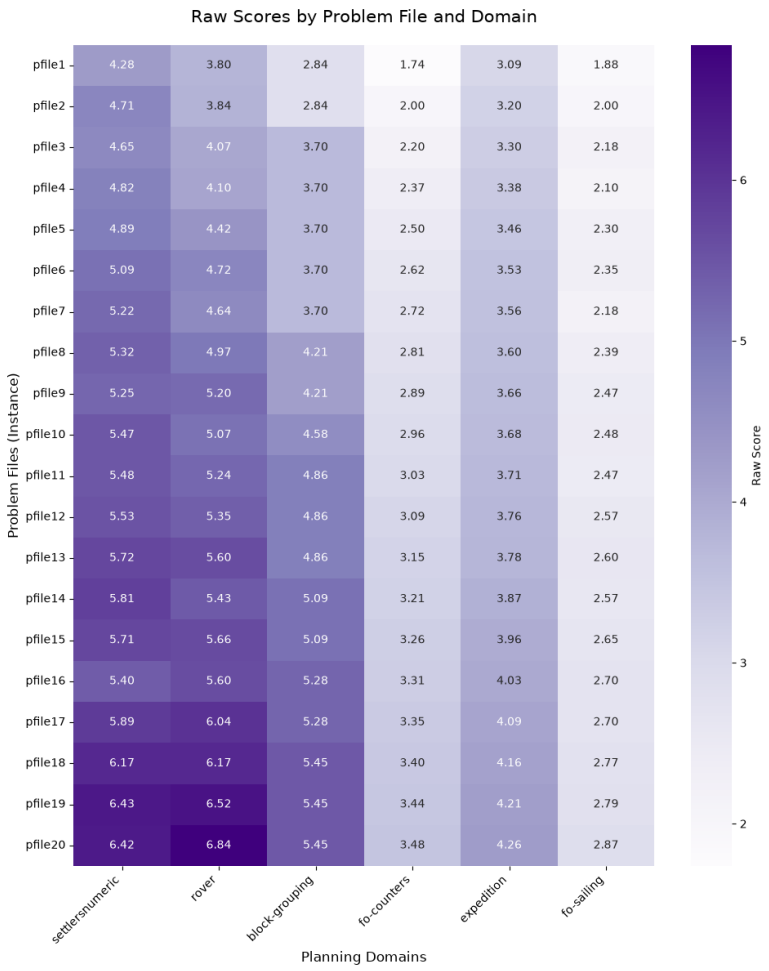
Numerical Optimization	Spatial	Logistic
Settlersnumeric	Block-Grouping	Rover
Sugar	Drone	Delivery
Tpp	Ext-Plant-Watering	Zenotravel
Pathwaysmetric	Sailing	Expedition
Markettrader	Fo-Sailing	
Fo-farmland		
Farmland		
Counters		
Fo-Counters		

Table 2: Reasoning Categories Selected for Benchmark

In order to obtain a more clear output from the LLM we decided to select two differents sets of three problems each and specifically:

- **Hard Set:** composed by *settlersnumeric, block-grouping, rover*
- **Easy Set:** composed by *fo counters, fo sailing, expedition*

It is clearly visible that we picked respectively the most three difficult problems and the three easiest ones. This has been done to ensure also in the outputs a visible difference in the metrics when evalauting the outputs. Additionally it can be seen from the following heatmap that the instances of these domains have increasing difficulty:



It is clearly visible both the difference in terms of difficulty between the Hard set and the Easy set and the increasing difficulty related to the pfiles; because of this last consideration we applied an additional, partition to divide the pfiles in three different categories and specifically:

- Easy (composed by pfiles from 1 to 4)
- Medium (composed by pfiles from 8 to 11)
- Hard (composed by pfiles from 15 to 18)

Also it is worth noticing that we did not use all the instances available for each selected domain but we decided to choose a small batch (4 instances) of instances representing the easy, medium and hard categories.

4 Configured LLMs

The benchmark framework was designed to support two execution backends: Leonardo HPC by CINECA and the NVIDIA API. Leonardo was used for local execution of Hugging Face models, while the NVIDIA API provided remote access to additional models without requiring every checkpoint to be installed and hosted locally.

4.1 Execution Backends

4.1.1 Leonardo HPC

Leonardo HPC was the main platform considered for local model execution. The models were loaded through the Hugging Face backend, allowing direct control over the execution environment, GPU resources, precision, prompt formatting, and generation parameters.

The selection of locally executable models was supported by whichLLM, a repository used to estimate which LLMs can run on a given hardware configuration and to compare models by release recency. This was useful to identify models that were realistic candidates for the available Leonardo resources before running the benchmark.

4.1.2 NVIDIA API

The NVIDIA API was used as a remote inference backend. This reduced the setup effort required to test recent open-weight models and made it possible to include models that would have been less convenient to install or run locally.

The benchmark pipeline remained the same across both backends: prompts, parsing, validation, and scoring followed the same workflow, while only the model adapter changed. The NVIDIA API backend also introduced practical constraints, such as rate limits, temporary service availability, and model-specific API key requirements.

4.2 Selection Criteria

The models were selected according to three practical criteria:

- **Hardware feasibility:** local models had to be compatible with the available Leonardo environment, using whichLLM as a practical reference for hardware requirements.
- **Recency:** priority was given to recent LLM families, using whichLLM as a reference to compare model release dates.
- **Model diversity:** the benchmark included models from different families to avoid evaluating a single architectural lineage.

Using both backends made the framework more flexible than a Leonardo-only setup. Leonardo provided controlled local execution, while the NVIDIA API allowed the benchmark to cover additional hosted models through the same interface.

4.3 Configured Models

The implemented model set includes the models configured for experimentation in the framework. Since the benchmark runs were still being refined, this section describes the configured model pool rather than treating any single run as the definitive final result.

Backend	Execution mode	Configured models
Leonardo HPC	Local HF	Gemma 4 31B, Phi-4, Nemotron Nano, Qwen 3.6
NVIDIA API	Remote hosted	Phi-4 Mini, GPT-OSS 120B, Kimi K2.6, Llama 3.3 70B, Nemotron Nano, Qwen 3.5

Table 3: LLM models configured in the benchmark framework.

5 Benchmark Output Parsing and Output Structure

The benchmark execution pipeline is designed to produce a set of traceable and independently analyzable artifacts. All generated data is stored within the `Benchmark_Framework/outputs` directory, which is partitioned to reflect the sequential stages of evaluation: generation, parsing, and scoring.

This separation of concerns is a core design principle. It allows for the independent diagnosis of different failure modes: a model might generate a correct plan in prose but fail due to a parsing error, or it might produce a syntactically valid but logically incorrect plan. By storing the output of each stage, we can distinguish between these cases.

The `outputs` folder is organized into three main subdirectories:

- **raw**: Contains the direct output from the language model.
- **parsed**: Stores the structured data extracted from the raw text.
- **scored**: Holds the final validation results and performance metrics.

Each of these directories follows a consistent hierarchical structure that mirrors the benchmark's matrix:

`<run_id>/<model>/<protocol>/<domain>/<difficulty>/<instance>.json`

This structure ensures that for any given problem instance, there are three corresponding folders: raw, parsed, and scored.

5.1 Raw

The files in the `outputs/raw/` directory are the most direct record of the interaction with the LLM. Each JSON file contains:

- The full prompt sent to the model, including the PDDL domain and problem definitions.
- The model's complete, verbatim text response.
- Any available "chain-of-thought" or reasoning text generated by the model before the final plan.
- Metadata from the model provider, such as generation tokens and stop reasons.

This layer serves as the ground truth for what the model produced, before any interpretation or processing.

5.2 Parsed

The `outputs/parsed/` directory contains the results of the first processing step. The parser attempts to extract a structured PDDL plan from the model's raw text response. Each JSON artifact in this layer includes:

- The extracted sequence of PDDL actions.
- The final plan formatted as a single block of text, ready for the validator.
- Any "chain-of-thought" actions if the model produced a separate reasoning plan.
- A record of any formatting issues or parsing errors encountered, which helps diagnose text extraction failures.

This layer bridges the gap between unstructured natural language and the formal PDDL syntax required for validation.

5.3 Scored

The final layer, `outputs/scored/`, contains the quantitative results of the evaluation. After a plan is parsed, it is passed to the external VAL validator and the resulting JSON files store:

- The official validation result from VAL.
- A boolean `solved` flag indicating whether the plan successfully achieves the goal.
- The number of iterations or repair attempts used to find a valid solution (if iterative repair is active).
- Metrics related to the plan's quality, such as its length and any specific errors identified by the validator.

6 Output Validation Using VAL

After the benchmark has obtained a textual answer from an LLM and extracted the candidate PDDL actions, the plan still has to be checked against the formal domain and problem definition. This is the role of VAL in the framework. The parser can say whether the model produced something that looks like an action sequence; VAL decides whether that sequence is actually executable and whether it reaches the goal.

6.1 Role of VAL in the Benchmark

VAL is used as an external symbolic validator. For each benchmark case, the framework provides VAL with three inputs:

- the PDDL domain file, containing predicates, actions, preconditions, and effects;
- the PDDL problem file, containing objects, initial state, and goal;
- the candidate plan extracted from the LLM output.

This separation is important because the benchmark does not rely on textual judgement to decide whether a plan is correct. A model can produce a plausible or well explained answer, but the answer is counted as solved only if the final action sequence validates against the corresponding PDDL task.

6.2 Validator Setup and Preflight

VAL is not a Python dependency of the project. It is an external executable that can be resolved in three ways: by putting `Validate` or `Validate.exe` on the system path, by passing an explicit executable path with `--validator-command`, or by using one of the bundled platform utilities under `utils/linux64/bin/` or `utils/win64/bin/`.

The benchmark can also run a task preflight before launching model generation. In this stage the framework checks the selected domain and problem files with VAL. This prevents malformed or unsupported PDDL inputs from being interpreted later as model failures. If VAL cannot be resolved, or if the preflight fails, the error is stored as an orchestration problem instead of being mixed with the LLM planning results.

6.3 Validation Flow

The validation path is the same for every model backend. The backend changes how the answer is generated, but not how the answer is checked:

$$\texttt{LLM output} \rightarrow \texttt{parsed actions} \rightarrow \texttt{temporary plan file} \rightarrow \texttt{VAL(domain, problem, plan)} \rightarrow \texttt{ValidationResult} \rightarrow \texttt{scored artifact}$$

The temporary plan file is created only to invoke VAL with the expected command line interface. After validation, the framework normalizes the validator output into a common result schema. This makes the rest of the benchmark independent from the exact textual format printed by the external validator.

6.4 Normalized Validation Result

The normalized validation record stores both the final decision and the diagnostic information needed for analysis and repair.

Field	Meaning
`valid`	Boolean result used to decide whether the candidate plan solved the task.
`status`	Normalized status such as `valid`, `invalid`, `parse_error`, `timeout`, or `validator_error`.
`error_type`	Compact failure class used by metrics and repair feedback.
`failed_step`	First failing step reported by VAL, when available.
`failed_action`	Problematic action extracted from the validator output, when available.
`goal_satisfied`	Whether the final state satisfies the goal.
`plan_length`	Number of action lines submitted to validation.
`validation_time_ms`	Time spent in the external validator call.
`raw_validator_output`	Original VAL output kept for debugging and later inspection.

Table 4: Main fields stored in the normalized validation result.

6.5 Prefix Validation

The framework does not validate only the full action sequence. It validates every non-empty prefix of the parsed final answer. This provides a more precise view of the model behaviour.

For example, a model may reach the goal after five actions and then continue with unnecessary or invalid actions. In that case, validating only the complete sequence would mark the attempt as invalid, while prefix validation reveals that a valid solution was present inside the answer. The scored artifact therefore records the first valid prefix length, the first valid plan text, whether the final full plan is valid, and how many extra actions appear after the first valid prefix.

This distinction is useful for the later reasoning analysis: it separates models that never find a valid plan from models that find a plan but fail to stop cleanly after satisfying the goal.

6.6 Failure Types and Repair Feedback

VAL failures are mapped into a small shared taxonomy. The most relevant classes are:

- `invalid_precondition`: an action is not applicable in the current state;
- `unsatisfied_goal`: the sequence executes but does not reach the required goal;

- `unknown_action`: the plan contains an action not defined in the domain;
- `syntax_error`: the plan format is not accepted by the validator;
- `timeout`, `validator_unavailable`, and `validator_crash`: technical validation failures, kept separate from planning mistakes.

In the iterative repair protocol, this normalized failure information is turned into feedback for the next model attempt. The feedback can include the validation status, the error type, the failed step, the failed action, and a compact description of the validator output. This makes the repair loop grounded in symbolic validation rather than in a generic request to try again.

6.7 Reasoning Validation

Some models expose a separate reasoning trace in addition to the final answer. The benchmark keeps the final answer as the official plan used for scoring, but it can also validate an action sequence extracted from the reasoning text. This second validation is diagnostic: it helps identify cases where the model derives a valid plan in its reasoning but fails to transfer it correctly into the final answer.

This distinction avoids giving credit for hidden reasoning when the final answer is invalid, while still preserving useful information for the reasoning capability analysis in the following section.

6.8 Metrics Derived from Validation

The validation result is the source for the core execution metrics. A case is considered solved only when the normalized validation result is valid. From this record the framework derives `validity_at_1`, `validity_at_k`, `repair_success`, `iterations_to_valid`, `plan_length`, `error_type`, and whether the run hit the iteration limit.

Therefore, VAL is not only a final correctness check. It is the component that connects raw model generations to comparable benchmark scores.

6.9 Limitations

The validation layer deliberately checks formal plan correctness, not the quality of the natural-language reasoning. It also depends on the availability and behaviour of an external executable, so timeout and validator errors are reported separately from genuine planning failures. Finally, the parser that normalizes VAL output uses conservative textual heuristics; for this reason the raw validator output is stored together with the normalized fields whenever manual inspection is needed.

7 LLMs Reasoning Capabilities Evaluation

7.1 Pipeline and Data flow

How Raw / Parsed / Scored become one DataFrame , which Classes hold the state the metrics need and why the load → row → aggregate → plot → report stages. This section maps the analysis script `advanced_planning_evaluation_sp.py` onto the rest of the Repo , showing how the three on-disk artifact layers Raw / Parsed / Scored. become DataFrames , which data classes exist and why and how the data-manipulation stage hands off to metric computation and visualization.

7.1.1 Pipeline Generic description

3	**1**	**8**	**27**	**7**
artifact layers	canonical DataFrame	aggregate tables	registered plots	capability profiles

7.1.2 The pipeline at a glance

Everything funnels into one DataFrame, `df_metrics` , the single source of truth for every `groupby` and every plot. Solid (teal) arrows are the data path; dashed (grey) arrows inject the PDDL ground truth needed to score plans (Figure 1).

7.1.3 How a row is built: the three inputs to `df_metrics`

The canonical table is produced by `compute_row_metrics()` . It does *not* read the filesystem and it does *not* compute ground truth — both happen before it. By the time the row loop runs, three things already exist, each with a structurally different role.

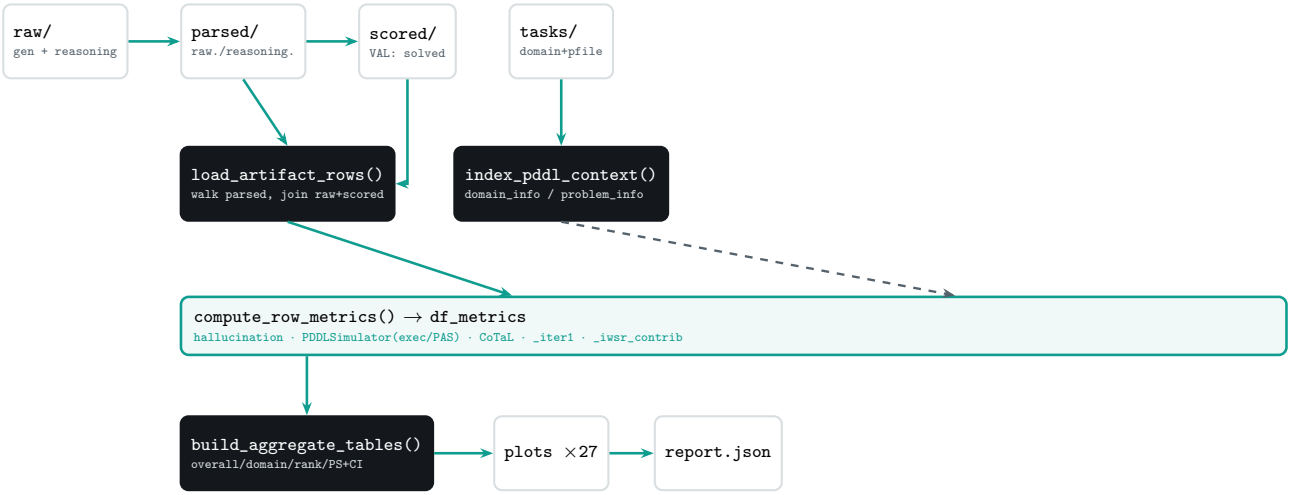


Figure 1: The analysis pipeline. One DataFrame, `df_metrics`, is the single source of truth; PDDL ground truth (dashed) is injected to score plans.

#	Input	Produced by	Role in the row loop
1	`df_raw`	`load_artifact_rows()`	The skeleton: one row per instance with the artifact-derived columns and the keys used to look up ground truth. It is `.copy()`-ed, then *widened* with computed columns.
2	`domain_info` / `problem_info`	`index_pddl_context()`	The ground truth: two dict lookups (`d_info`, `p_info`) consumed per row to score the plan. *Not* stored as columns — they are the reference the metric functions compare against.
3	`warnings_out`	threaded list via `add_warning()`	The diagnostic side-channel: a by-reference list accumulating structured records whenever an input is missing or malformed. Never part of `df_metrics`; embedded separately in the report.

What `df_metrics` literally is

`df_metrics = df_raw.copy()` horizontally concatenated with three blocks of computed columns — hallucination, precondition/executability (from `PDDLSimulator`), and CoTaL — plus two derived columns `_iter1` and `_iwsr_contrib`. So input 1 supplies the columns you keep; input 2 is *consumed* to manufacture the new columns and then discarded; input 3 is a parallel log, not a column. The "three components" mental model is right, but only the first is literally part of the frame.

`df_raw` — the artifact skeleton `load_artifact_rows()` walks the canonical path `parsed/<run>/<model>/<protocol>/<domain>/<difficulty>/<instance>.json` and, for each instance, joins the matching `scored/` data. It emits exactly *one row per problem instance*, collapsing the retry loop by keeping the last non-empty plan as the representative `_actions` while preserving the full attempt lists as private columns.

```
# public  -- identity + outcome
Model, Domain, Problem, Difficulty, Protocol, Run_id
Valid (= scored["solved"]), Length, Iterations (= scored["iterations_used"]), Chain_of_Thought
# private -- consumed downstream, dropped from the report
_actions          # representative plan (last non-empty attempt)
_cot_text         # reasoning trace text
_parsed_attempts  _scored_attempts  _raw_attempts   # full per-iteration lists
_file_path  parsed_file_path
```

The keys that matter for the next input are `Domain`, `Difficulty` and `Problem` — together the exact triple used to retrieve per-instance ground truth. The `Chain_of_Thought` flag is resolved from three sources (protocol YAML, presence of reasoning text, presence of reasoning actions) so CoT-bearing runs are never mislabelled.

Why one row per instance, not per attempt

Keeping the grain at the *instance* level means every aggregate (success rate, hallucination, PS) is automatically an average over comparable units. The iteration structure is not lost — it survives in the private

`_*_attempts` columns and the `Iterations` count — but it is opt-in, read only by metrics that need it (FASR, IWSR, the iteration profile, CoTaL).

The PDDL parsing layer — `d_info` and `p_info` Plans can only be scored against the formal task they were generated for, so the framework parses the PDDL itself. Two small, deliberately permissive regex parsers do this — a syntax error emits a warning, never aborts the run. They answer two different questions and return two differently-shaped dicts.

`parse_domain_pddl(domain_path)` reads one `domain.pddl` (the schema shared by every instance in a domain) and returns the legal vocabulary plus per-action templates:

```
parse_domain_pddl(...) -> {
  "action_names": set[str],   # every legal action head -- the hallucination whitelist
  "predicates":   set[str],   # legal predicate names
  "functions":    set[str],   # numeric fluent names -- what is numeric, not propositional
  "schemas": {                # one entry PER action, the simulator's instruction sheet
     action_name: {
        "params":   [str, ...],   # ordered ?param variable names, for grounding
        "prec_raw": str,          # raw :precondition s-expression text (unparsed)
        "eff_raw":  str,          # raw :effect s-expression text (unparsed)
     }, ...
  },
  "path": str | None,         # None when the file was missing (empty stub)
}
```

Each field has a single downstream consumer: `action_names` is the whitelist for action hallucination; `functions` tells the simulator which heads are numeric fluents; and `schemas` is what the simulator grounds — `params` maps the action's arguments onto the variables inside the raw `prec_raw` / `eff_raw` strings so preconditions can be checked and effects applied.

`parse_problem_pddl(problem_path)` reads one instance file (its own object set and starting state) and returns:

```
parse_problem_pddl(...) -> {
  "objects":      set[str],               # legal argument tokens -- object-hallucination whitelist
  "init_atoms":   set[tuple[str, ...]],   # initial true propositions, e.g. ("at","truck1","depot")
  "init_numeric": dict[tuple, float],     # initial fluent values, e.g. ("fuel","truck1") -> 50.0
  "path":       str,
  "difficulty": str,                      # parent folder name
}
```

Object hallucination uses `objects`; the simulator seeds its world from `init_atoms` and `init_numeric`. Action- vs object-hallucination are kept as separate signals on purpose — a model can know the legal action names yet still invent objects, a different and shallower grounding failure.

`index_pddl_context(tasks_dir)` calls those two parsers exactly once each per file, walking `tasks/` a single time, and packs the results into two lookups whose key shapes mirror the granularity of what they hold:

```
domain_info  : dict[str, dict]                 # key = domain_name  (schema is shared)
problem_info : dict[tuple[str,str,str], dict]  # key = (domain, difficulty, instance)
```

This is the "parse once, consume many" design: every PDDL file is read once regardless of how many rows or retries reference it. In the row loop the retrieval is then two O(1) dict gets:

```
d_info = domain_info.get(domain, {})                          # by domain
p_info = problem_info.get((domain, difficulty, instance), {})   # by full triple
pddl_ok = bool(d_info.get("action_names")) and bool(p_info.get("objects"))
```

A missing domain and a missing problem fail differently

When `domain.pddl` is absent, `index_pddl_context` still stores a populated-but-empty stub under the domain key (empty `action_names`, empty `schemas`, `path=None`). When a problem file is absent, no entry is created and the row's `.get(..., {})` returns a bare `{}`. Both end up empty, but only the

domain case leaves a record — and an empty `action_names` set means *every* emitted action looks illegal, silently depressing that domain's grounding numbers. The only trace is a warning, which is exactly what input 3 is for.

`warnings_out` — the diagnostic accumulator `warnings_out` is a single `list[dict]` created once at the top of the analysis run and threaded by reference into every stage that can encounter a malformed or missing input — `index_pddl_context`, both PDDL parsers, and `compute_row_metrics`. Each stage appends through one helper:

```
def add_warning(warnings_out, warning_type, message, **extra):
    warnings_out.append({"type": warning_type, "message": message, **extra})
    # **extra carries locating context: run_id, domain, difficulty, problem, path
```

Its purpose is to replace scattered `print` / `logging` calls with one structured, serialisable trace that travels with the results: the full list is embedded in `report.json` under the `warnings` key, so a notebook or CI job can filter by `type` and locate the offending artifact without re-running anything. Typical entries are `missing_tasks_dir`, `missing_domain_pddl`, `missing_pddl_context`, `domain_parser_error`, `problem_parser_error`. Rows with incomplete context are *not* dropped — they still get NaN metric values — so the instance count stays honest while the warning records why those NaNs exist.

One diagnostic path bypasses `warnings_out`

The simulator is the exception. If `compute_precondition_metrics` throws, it falls back to a NaN result and reports through Python's stdlib `warnings.warn("[simulator] ...")` — *not* through `add_warning`. So a simulator crash does not appear in the structured `warnings` list in the JSON report; it surfaces only on stderr. Worth unifying if the report is to be the single audit log.

7.1.4 The merge: three inputs into one widened frame

`compute_row_metrics` iterates `df_raw` once. For each row it resolves `d_info` / `p_info` (input 2), runs the three metric families against the row's `_actions` and attempt lists, and emits any incompleteness into `warnings_out` (input 3). The per-family results are collected and concatenated columnwise back onto the copied skeleton.

```
df = df_raw.copy()
for _, row in df.iterrows():
    d_info  = domain_info.get(row["Domain"], {})
    p_info  = problem_info.get((row["Domain"], row["Difficulty"], row["Problem"]), {})
    if not (d_info.get("action_names") and p_info.get("objects")):
        add_warning(warnings_out, "missing_pddl_context", ...)        # input 3
    halluc_rows.append(      compute_hallucination_metrics(actions, d_info, p_info))
    precondition_rows.append(compute_precondition_metrics(actions, d_info, p_info))  # PDDLSimulator
    cot_rows.append(         ... compute_cot_alignment_for_attempt(...) ...)

df_metrics = pd.concat([df, halluc_df, precondition_df, cot_df], axis=1)
df_metrics["_iter1"]        = (df.Valid == True) & (df.Iterations == 1)   # -> FASR
df_metrics["_iwsr_contrib"] = 1/Iterations if Valid else 0                # -> IWSR
```

The result, `df_metrics`, is the single source of truth every `groupby` in `build_aggregate_tables` and every plot reads from. Note the grain: the hallucination and executability columns describe the *representative* (last non-empty) plan, while the CoTaL and efficiency signals reach back into the preserved attempt lists.

7.1.5 Data classes that make the metrics possible

Two classes carry state that a flat function could not — introduced here, with the simulator expanded in full in Section 7.1.6.

PDDLSimulator (in the analysis script). A forward STRIPS + simple-numeric simulator written in pure Python so the analysis layer needs no VAL subprocess. Instantiated *once per (problem, plan)* inside `compute_precondition_metrics`. Its decisive feature is `prop_hist`: a list of `frozenset` snapshots of the propositional state after every applied step. That history is what lets the framework, at the first failing action,

ask *"was this precondition ever true earlier?"* — the question separating a sequencing error (true before, deleted too early) from a state fabrication (never true). Without the per-step history there is no PAS and no temporal distance.

RunMetrics (`evaluators/metrics.py`, `@dataclass`). The validator-side metric bundle written at scoring time: `validity_at_1`, `validity_at_k`, `repair_success`, `iterations_to_valid`, `plan_length`, `error_type`, `hit_iteration_limit`. The analysis script re-derives most of these from the scored JSON, but this dataclass is the contract guaranteeing every model is scored on the same basis, independent of backend.

The error vocabulary is fixed in `evaluators/error_taxonomy.py` as an `ErrorType` enum, partitioned into `LOGICAL_ERROR_TYPES` (empty plan, syntax, unknown action, invalid precondition, unsatisfied goal) versus `TECHNICAL_ERROR_TYPES` (parse error, timeout, validator crash/unavailable). That split keeps infrastructure failures from being charged against a model's planning score.

7.1.6 `PDDLSimulator` in depth

Almost every execution-side discriminator — executability ratio, the sequencing-vs-fabrication split, PAS, and mean temporal distance — is produced by one small class. It is a **forward STRIPS + simple-numeric simulator written in pure Python**, deliberately so: the analysis layer can re-derive these signals without spawning a VAL subprocess or shipping a compiled binary. It reproduces just enough of the validator's logic for the propositional and numeric-inequality fragments the benchmark uses, trading completeness for portability.

What it is for. The class does three things: it **holds a world state** (a set of true propositions plus a dict of numeric fluents), it **walks the plan one action at a time** checking each action's preconditions before applying its effects, and — its decisive feature — it **saves a frozen snapshot of the state after every applied step**. That snapshot history is what makes the precondition-awareness metrics computable. It is the basis for four metrics: executability ratio (how far the plan runs before the first break), the sequencing-error / state-fabrication counts, PAS (their ratio), and mean temporal distance. It targets *forward*, *propositional* and *simple-numeric* domains — STRIPS add/delete effects plus `increase` / `decrease` fluents and $\geq \leq > <$ comparisons — and is invoked exactly once per `(problem, plan)` pair.

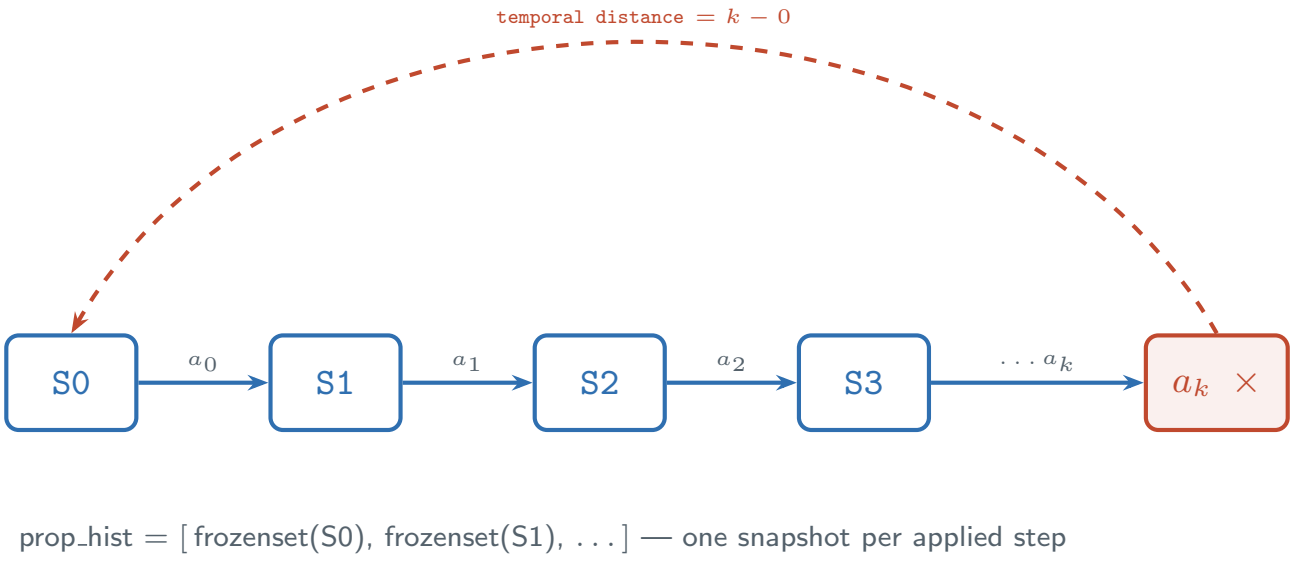


Figure 2: State evolution. At the first failing action a_k, each unmet precondition atom is traced back through `prop_hist`: found earlier → sequencing error (record k − last as temporal distance); never found → state fabrication.

State, snapshots, and the step loop. The constructor seeds the world from the parsed problem and takes the first snapshot before any action runs; `prop_hist` then grows by one frozen set per applied step.

```
def __init__(self, d_info, p_info):
    self.prop      = set(p_info.get("init_atoms", set()))   # true propositions
    self.num       = dict(p_info.get("init_numeric", {}))   # numeric fluents
    self.prop_hist = [frozenset(self.prop)]                 # snapshot S0 = initial state
```

```
    self.step      = 0
```

For each action, `compute_precondition_metrics` grounds the schema, checks preconditions, and — only if they hold — applies the effects and appends a new snapshot. The walk **stops at the first failing action** (a `break`), which makes "how far did it get" a meaningful prefix length.

```
for step, action_str in enumerate(actions):
    name, args = parse_action(action_str)
    schema = schemas.get(name)
    if name is None or schema is None:        # unparseable / unknown action: skipped
        continue
    if len(schema["params"]) != len(args):    # wrong arity -> fabrication, stop
        first_failure = step; fab_errors += 1; break
    ok, failed_prop, failed_num = sim.check_preconditions(schema["prec_raw"], params, args)
    if not ok:
        first_failure = step
        for atom in failed_prop:
            last = sim.last_step_atom_true(atom)         # search the snapshot history
            if last >= 0: seq_errors += 1; temporal_distances.append(step - last)
            else:         fab_errors += 1
        fab_errors += len(failed_num); break
    sim.apply_effects(schema["eff_raw"], params, args)   # mutate state + append snapshot
```

The classification logic, atom by atom. At the failing action, each unsatisfied propositional precondition is the unit of analysis. `last_step_atom_true(atom)` scans `prop_hist` newest-to-oldest and returns the latest snapshot index at which the atom held — or `-1` if it never did.

```
def last_step_atom_true(self, atom):
    for index in range(len(self.prop_hist) - 1, -1, -1):
        if atom in self.prop_hist[index]:
            return index
    return -1
```

That single lookup is the whole basis of the sequencing/fabrication distinction. If the atom was true at some earlier snapshot, the model had the prerequisite and then ordered actions so as to destroy it — a **sequencing error**, and the gap step − last is recorded as a temporal distance. If the atom was never true, the model required a world state that never existed — a **state fabrication**. Effects are applied in STRIPS order (deletes before adds), so the snapshots faithfully reflect what each action left behind. The metrics fall straight out of these counts:

```
executability_ratio = first_failure / plan_len
PAS = sequencing_errors / (sequencing_errors + state_fabrications)
mean_temporal_distance = mean(step - last_true_step for each sequencing error)
```

The counts describe the first failure, not the whole plan

Because the loop `break`s at the first failing action, the sequencing/fabrication counts and the temporal distances come from that *one* action's unmet preconditions — PAS is a property of where the plan first breaks, not a tally over the entire sequence. This is intended (it localises the failure), but it means a plan can accrue at most one failure event in these columns.

Unknown actions are skipped, which inflates executability

An action whose name is not in the schema is `continue`d over — it neither advances the state nor counts as a failure here (name-level hallucination is scored separately). So a plan made entirely of hallucinated action names never triggers a precondition failure and records `executability_ratio = 1.0`. Always read ER alongside the hallucination columns. Relatedly, because skipped actions advance the plan index but not the snapshot index, temporal distance is measured in generated-plan positions relative to the snapshot where the atom last held — the two clocks coincide only when every prior action was a known, applied schema.

7.1.7 From data to metrics to pictures — why each stage exists

The architecture separates concerns on purpose, so each is independently testable.

Stage	Function	Purpose / why it is separate
Load	`load_artifact_rows`	Decouples the filesystem walk from metric logic; everything below receives a uniform table.
Row metrics	`compute_row_metrics`	One pass attaches every per-instance metric; private `_actions` / `_cot_text` stay alive only here.
Aggregate	`build_aggregate_tables`	All `groupby` logic in one place so the JSON report can embed tables whether or not plots are drawn.
Visualise	`maybe_make_plots`	Reads only the aggregate tables; registering each figure embeds its path + description into the report.
Assemble	`build_model_payloads`	Re-keys everything by *model* so a consumer can pull one model in O(1), and attaches the capability-profile classification.

A real artefact to know about

Because `compute_row_metrics` keeps the last non-empty plan as the representative for the instance-level hallucination/executability columns, those columns describe the *final* attempt. Metrics that must see all attempts (FASR, IWSR, CoTaL per-iteration, the iteration profile) read the preserved `_*_attempts` lists instead. When reading a plot, always check which grain it is on — this is spelled out per metric in the following sections.

7.2 Metrics scientific foundation

The ten papers that license the metric set — each pinned to the exact paragraph, table, figure or definition it is taken from, with numbers, paraphrased, linked, and mapped to the design decision it grounds; plus the literature-motivated metrics not yet implemented.

The framework's thesis is that binary validity is not enough: two models can both fail and be failing for completely different reasons. Every metric isolates one mechanism, and each is anchored to a specific result in the literature. Each card below pins the *exact* location the claim is taken from, paraphrases what that spot reports (with its numbers), states what it grounds here, and links to the source.

The one-line argument

If LLM planning is closer to approximate retrieval than to search (Kambhampati 2024; Valmeekam 2023), then (a) retries are near-independent samples so success must be discounted by retry count — *FASR, IWSR* ; (b) feedback content is largely irrelevant so a large retry gap is not correction — *Stechly 2023* ; and (c) the only way to separate retrieval from reasoning is to probe execution, grounding and reasoning-faithfulness directly — *exec_ratio, PAS, hallucination, CoTaL* .

Provenance and a corrected citation

Locations, page numbers and quantitative results below follow the project's verified reference apparatus. The Critical Investigation is cited from the NeurIPS 2023 proceedings (the version of record) with preprint arXiv:2302.06706 — this supersedes the arXiv:2305.15771 id used in an earlier pass. The 2022 benchmark ("Still Can't Plan", arXiv:2206.10498) and the 2025 frontier study (Correa et al., arXiv:2511.09378) are added here. Findings are paraphrased; quoted fragments are short technical phrases; numbers are reported as facts.

REFERENCE: Large Language Models Still Can't Plan (A Benchmark for LLMs on Planning and Reasoning about Change)

Valmeekam, Olmo, Sreedharan &Kambhampati (2022) · NeurIPS 2022 FMDM Workshop arXiv:2206.10498 ↗ DOI 10.48550/arXiv.2206.10498

Where in the paper §5" Current Curriculum for Testing", the seven test cases (pp. 6–8); §5.2 Optimal Planning + Table 1; §7 Conclusion (p. 9).

What that location reports Introduces a seven-task curriculum — plan generation, cost-optimal planning, reasoning about plan execution, robustness to goal reformulation, plan reuse, replanning, plan generalisation — precisely because plan-validity alone captures only one dimension; only the first two

are "actual planning", the rest are auxiliary reasoning-about-change tasks. InstructGPT-3 solved ˜ 3.2% of optimal-planning vs ˜ 5% of plan-generation instances (valid ≠ efficient). The conclusion explicitly lists adding a partial-correctness metric as future work.

What it grounds here The original literature justification for going beyond pass/fail — the framework's whole thesis. Motivates the (not-yet-implemented) partial-credit / Goal-Achievement score and an Optimality Ratio.

Read it Open the source ↗

REFERENCE: On the Planning Abilities of Large Language Models — A Critical Investigation

Valmeekam, Marquez, Sreedharan &Kambhampati (2023) · NeurIPS 2023 NeurIPS 2023 · arXiv:2302.06706 ↗ DOI 10.48550/arXiv.2302.06706

Where in the paper §2 Related Work para.5 (p. 3) Phase 1/Phase 2; §4 Autonomous Mode" Obfuscating names..."(pp. 7–8) + Table 1; §4 Figure 3 (p. 7) relaxations; §5.2" Verifier-assisted repeated backprompting", Table 4 (p. 9); Table 1 CoT rows (p. 5).

What that location reports Splits planning into Phase 1 (discovering actions and their precondition/effect dependencies) and Phase 2 (sequencing them without harmful interactions); LLMs are strong on Phase 1, weak on Phase 2. Renaming Blocksworld into Mystery Blocksworld collapses GPT-4 zero-shot from 210/600 (35%) to 1/600 (0.16%) — "pattern matching rather than reasoning". Figure 3 classifies failures as inexecutable vs executable-but-goal-not-reached. Backprompting (≤15 rounds) eventually solves 82% Blocksworld / 70% Logistics at avg 3.68 / 3.31 rounds, but only 10% in Mystery Blocksworld. CoT barely helps (Blocksworld 210→214/600; Mystery 1→54/600).

What it grounds here Grounds the exec_ratio + PAS pair (Figure 3's classification, refined into sequencing vs fabrication), the hallucination gate (obfuscation collapse), FASR (pre-backprompt performance), cross-domain consistency (the obfuscation gap), the Lucky-Retriever profile, and CoT as diagnostic not performative.

Read it Open the source ↗

REFERENCE: GPT-4 Doesn't Know It's Wrong: An Analysis of Iterative Prompting for Reasoning Problems

Stechly, Marquez &Kambhampati (2023) · NeurIPS 2023 FMDM Workshop arXiv:2310.12397 ↗ DOI 10.48550/arXiv.2310.12397

Where in the paper Abstract, findings (i)–(iii); Abstract para.2 (the complexity-argument paragraph).

What that location reports On graph colouring GPT-4 is bad at solving and no better at verifying, so LLM-critiques-LLM loops do not help; crucially the content of the criticisms — from an LLM or an external solver — is "largely irrelevant" to iterative-prompting performance. The intuition that verification is easier than generation is a complexity argument that does not apply to approximate retrieval. Apparent iterative gains come from the correct answer fortuitously appearing across repeated independent samples.

What it grounds here The keystone of the retry apparatus: if feedback is irrelevant, each retry is an independent re-sample → a large Retry Gap is stochastic search, IWSR discounts late wins, and a flat P(Valid—k) is the Stochastic-Searcher signature.

Read it Open the source ↗

REFERENCE: Can Large Language Models Reason and Plan?

Kambhampati (2024) · Annals of the New York Academy of Sciences 1534(1):15–18 arXiv:2403.04121 ↗ DOI 10.1111/nyas.15125

Where in the paper para.4–7 the two-phase model; pp. 3–4 the self-critique paragraphs; p. 2 the approximate-retrieval / CoT paragraph.

What that location reports Distinguishes having planning-domain knowledge (Phase 1) from assembling it into an executable plan that handles subgoal interactions (Phase 2). What LLMs do is "universal approximate retrieval", not principled reasoning; self-critique is of questionable utility because they hallucinate both false positives and false negatives when verifying their own solutions. Hence the fake-reasoning vs genuine-deliberation distinction.

What it grounds here The theoretical organiser of the failure-mode quadrant; the justification for IWSR penalising late convergence; and the SR-high + CoT-low "fake reasoning" diagnosis.

Read it Open the source ↗

REFERENCE: Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents

Huang, Abbeel, Pathak &Mordatch (2022) · ICML 2022 · PMLR 162:9118–9147 arXiv:2201.07207 ↗ DOI 10.48550/arXiv.2201.07207

Where in the paper §2.2" Metrics" and Figure 1 (pp. 2–3).

What that location reports First formal separation of executability from correctness as two independent axes: large models (GPT-3 175B, Codex 12B) produce plans with high semantic correctness but low executability — as low as ˜ 18% for raw Codex. A wander-around program is highly executable but incorrect; a human-written plan is correct but often inexecutable — so both axes are needed.

What it grounds here The direct basis for exec_ratio as a metric separate from Success_Rate; the framework's exec_ratio is a continuous, finer-grained version of Huang's binary executability.

Read it Open the source ↗

REFERENCE: Measuring Faithfulness in Chain-of-Thought Reasoning

Lanham, Chen, Radhakrishnan et al. (2023) · Anthropic (2023) arXiv:2307.13702 ↗ DOI 10.48550/arXiv.2307.13702

Where in the paper Abstract + §3 (the Early-Answering intervention and post-hoc-reasoning definition).

What that location reports Truncating the chain of thought before the answer tests whether the stated reasoning is causal or post-hoc; models vary widely in how much they condition on the CoT, and faithfulness tends to decrease as models scale.

What it grounds here CoTaL operationalises faithfulness as content/structure overlap between the reasoning trace and the emitted plan; the SR-high + CoT-low = fake-reasoning diagnosis is the direct application.

Read it Open the source ↗

REFERENCE: Embers of Autoregression: Understanding LLMs Through the Problem They Are Trained to Solve

McCoy, Yao, Friedman, Hardy &Griffiths (2024) · PNAS (2024) arXiv:2309.13638 ↗ DOI 10.48550/arXiv.2309.13638

Where in the paper Abstract (cipher example); §5 sensitivity to task probability; §6 sensitivity to output probability; Table 1.

What that location reports Even on deterministic tasks, accuracy tracks the probability of the task / output / input. GPT-4 decodes a shift cipher ˜ 51% of the time when the answer is high-probability but only ˜ 13% when low-probability.

What it grounds here Grounds the FASR-by-difficulty analysis: a sharp easy→hard cliff signals sensitivity to training-data frequency, not logical complexity.

Read it Open the source ↗

REFERENCE: Faith and Fate: Limits of Transformers on Compositionality

Dziri, Lu, Sclar et al. (2023) · NeurIPS 2023 (Spotlight) arXiv:2305.18654 ↗ DOI 10.48550/arXiv.2305.18654

Where in the paper The computation-graph formulation of compositional tasks and the error-by-depth analysis.

What that location reports Multi-step tasks (multi-digit multiplication, a logic puzzle, a DP problem) are cast as computation graphs; transformers reduce multi-step reasoning to "linearised subgraph matching", with success tied to having seen graph fragments in training and accuracy decaying as compositional depth grows (e.g. ˜ 59% on 3×3 multiplication).

What it grounds here Grounds the temporal-distance reading and the exec-vs-length decay slope (local steps fine, long-horizon sequencing collapses) — and the Understander profile, where the collapse depth is itself measurable.

Read it Open the source ↗

REFERENCE: Evaluating the World Model Implicit in a Generative Model

Vafa, Chen, Rambachan, Kleinberg &Mullainathan (2024) · NeurIPS 2024 (Spotlight) arXiv:2406.03689 ↗ DOI 10.48550/arXiv.2406.03689

Where in the paper Abstract + §2: the DFA / Myhill-Nerode framework, the MNI and MNB constructions, and the two metrics.

What that location reports Treats a deterministic domain (PDDL planning is a DFA) and asks

whether a model's implicit world model is coherent. Two states are equivalent iff all future sequences agree; the Myhill-Nerode interior (MNI) is the set of continuations valid from both states, the boundary (MNB) the minimal suffixes that separate them. From these: a compression score (do same-state histories yield an empty implied boundary?) and a distinction score (does the model reproduce the true boundary between different states?). Models look fine on standard diagnostics yet their world models are "far less coherent than they appear".

What it grounds here Grounds the Vocabulary-Only profile (high exec / low PAS = surface fluency over an incoherent world model) — and is the strong form of which PAS is explicitly the weak version.

Read it Open the source ↗

REFERENCE: The 2025 Planning Performance of Frontier LLMs (rev.: Frontier LLMs Rival State-of-the-Art Planners)

Correa, Pereira et al. (2025) · preprint (2025) arXiv:2511.09378 ↗ DOI 10.48550/arXiv.2511.09378

Where in the paper Abstract, results tables and Figure 1; eight domains of the IPC 2023 Learning Track; plans VAL-verified.

What that location reports The most directly comparable contemporary benchmark. Evaluates DeepSeek R1, Gemini 2.5 Pro and GPT-5 against the classical planner LAMA; on standard PDDL tasks GPT-5 is competitive with LAMA. Under obfuscation (predicate/action names stripped of semantics) every LLM degrades while LAMA is invariant to renaming (GPT-5 152, Gemini 2.5 Pro 146, DeepSeek R1 93 solved). The expanded version adds Gemini 3/3.1 Pro (245 vs 234 baseline) and notes some obfuscated re-identification, hinting performance is not wholly reducible to training exposure.

What it grounds here Its Figure 1 is essentially the target plot. Introduces the standard-obfuscated gap as a metric (small gap = structural reasoning, large gap = semantic exploitation) — a cleaner operationalisation of the hallucination/grounding axis, and endorses VAL verification + freshly-generated tasks against contamination.

Read it Open the source ↗

7.2.1 Claim → paper → location → support

The spine of the design: each implementation decision tied to the precise spot that licenses it, with the supporting result paraphrased.

7.2.2 Literature-motivated metrics beyond the current set

The same papers also point to metrics the framework does not yet implement. Listed here as honest future directions, not as shipped features.

On DOIs and versions of record

arXiv preprints resolve through 10.48550/arXiv.
¡
id
¿
; Kambhampati 2024 (Annals NYAS, 10.1111/nyas.15125) and McCoy 2024 (PNAS) also carry publisher DOIs. The Critical Investigation links to the NeurIPS proceedings PDF; Huang 2022 is canonically PMLR 162 (preprint arXiv:2201.07207). Correa et al. 2025 (arXiv:2511.09378) has a v1 title ("The 2025 Planning Performance...") and an expanded later version ("...Rival State-of-the-Art Planners"); both share the id.

7.3 Metric Families

7.3.1 Planning Efficiency

Success Rate, First-Attempt Success Rate, Iteration-Weighted Success Rate, Retry Gap — and the iteration profile that distinguishes genuine correction from stochastic resampling.

This family answers *"how often, how cheaply, and how honestly does the model succeed?"* All four metrics live in the `overall` / `by_domain` tables and share one design principle: a success that needed many retries is worth less than a success on the first try, because under the retrieval hypothesis those retries are independent samples, not corrections.

Framework claim	Paper	Location	Support
Binary validity is insufficient	Valmeekam 2022	§5 seven-task curriculum	Only 2 of 7 tasks are "actual planning"
Partial credit / optimality needed	Valmeekam 2022	§7; §5.2 Table 1	Partial-correctness flagged as future work; 3.2% optimal vs 5% valid
Hallucination gate threshold	Valmeekam 2023	§4 Obfuscation, Table 1	GPT-4 35% → 0.16% under name obfuscation
Phase 1 / Phase 2 distinction	Valmeekam 2023	§2 Related Work, p. 3	discovering actions vs sequencing a subset
exec_ratio + PAS pair	Valmeekam 2023	§4 Figure 3	inexecutable vs goal-not-reached classification
FASR as honest metric	Valmeekam 2023	§5.2 Table 4, p. 9	82%/70% eventual, avg 3.68/3.31 rounds; Mystery 10%
Lucky-Retriever profile	Valmeekam 2023	§4 Obfuscation	"pattern matching rather than reasoning"
Stochastic Searcher / retry gap	Stechly 2023	Abstract, finding (iii)	criticism content "largely irrelevant"
IWSR penalises late retries	Stechly 2023	Abstract para.2	approximate retrieval voids the complexity argument
Fake reasoning (SR high + CoT low)	Kambhampati 2024	p. 2 approximate retrieval	retrieval vs deliberation
exec &SR independent axes	Huang 2022	§2.2 Metrics, Figure 1	executability/correctness trade-off
CoT alignment metric	Lanham 2023	Abstract + §3	CoT faithfulness operationalised
Cliff drop easy → hard	McCoy 2024	Abstract + §5–6	performance follows training probability
High exec + low PAS = incoherent world model	Vafa 2024	Abstract + §2 (MNI/MNB)	surface fluency without a coherent world model
Long-horizon collapse (Understander)	Dziri 2023	computation-graph error-by-depth	reasoning decays with compositional depth
Standard - obfuscated gap	Correa 2025	Results, Figure 1	LLMs degrade under renaming; LAMA invariant

Real values from the bundled run

gpt_oss_120b is the clear leader: SR
0.524, FASR
0.333, IWSR
0.371, Retry
Gap
0.190. qwen_3_5_122b shows the diagnostic pattern that motivates the whole family — SR
0.500 but FASR
0.000 and Retry
Gap
0.500: every success arrived only after retries, the signature of a Stochastic Searcher.

METRIC · SR: Success Rate

overall[" Success_Rate"] = mean(Valid) Fraction of instances the model ever solves, across all allowed iterations. The headline number — and the one the framework argues is least trustworthy on its own.

How it is computed Boolean `Valid` (from `scored["solved"]`) averaged per group. This is *validity@k*: success anywhere in the repair budget counts.

```
Success_Rate=("Valid", "mean")
```

Range & meaningful bands Bands used by the profile thresholds: ≈0 → No Grounding ·¡0.2 → Un-

Candidate metric	Source	What it would add
Goal-Achievement / partial-credit score	Valmeekam 2022 §7	Fraction of goal atoms achieved, instead of all-or-nothing validity — the benchmark authors' own stated gap.
Optimality Ratio	Valmeekam 2022 §5.2	Plan length / optimal length, so a long valid plan is not scored equal to a minimal one.
Standard - obfuscated gap	Correa 2025	Per-model delta between named and renamed domains — a direct, cleaner grounding measure than hallucination rate alone.
Strong world-model score (MNI/MNB)	Vafa 2024	Compression + distinction scores over shared mid-plan sub-states; PAS is the weak, single-step version of this.

derstander/Vocabulary ·¿0.5 capable ·¿0.7 strong. Always read against FASR; SR alone cannot distinguish retrieval from reasoning.

In the plots `success_rate_heatmap` (model×domain), the SR bars in `sr_vs_fasr` and `sr_fasr_iwsr_by_model`, and it drives the colour in `failure_mode_taxonomy` and `fasr_iwsr_scatter`.

Rationale SR is the necessary baseline but the framework's thesis is that it is *insufficient*: it conflates first-try success with retry-dependent success and says nothing about *why* a failure happened. Every other metric exists to decompose it.

METRIC · FASR: First-Attempt Success Rate

overall[" FASR"] = mean(_iter1) Fraction solved on iteration 1 with no feedback at all. The framework treats this as the single honest signal of planning ability, because it is the only one that cannot be inflated by retries.

How it is computed A private boolean column is set at row level and averaged:

```
df["_iter1"] = (df["Valid"] == True) & (df["Iterations"] == 1)
FASR=("_iter1", "mean")
```

Range & meaningful bands FASR¿0.50 with low Retry Gap → genuine zero-shot capability. FASR ≈ 0 while SR is high → the model only ever succeeds by resampling (the entire qwen / nemotron pattern in the bundled run).

In the plots `fasr_by_model_domain`, `fasr_by_difficulty`, the FASR bar in `sr_vs_fasr` / `sr_fasr_iwsr_by_model`, both axes of `fasr_iwsr_scatter`, and the y-axis of `failure_mode_taxonomy` bubble size.

Rationale Under the retrieval hypothesis (Kambhampati 2024) and the irrelevant-feedback finding (Stechly 2023), only the un-retried attempt reflects what the model can do in one shot. FASR carries the highest weight in the composite score (0.25) for exactly this reason.

METRIC · IWSR: Iteration-Weighted Success Rate

contrib = (1/Iterations) if Valid else 0 Success discounted by how many attempts it took. Rewards models that converge fast; penalises models that only succeed deep into the retry budget.

How it is computed Each solved instance contributes `1/k` where k is the iteration count; unsolved contributes 0. The mean of these contributions is IWSR.

```
df["_iwsr_contrib"] = (1.0/row["Iterations"]) if row["Valid"] and row["Iterations"]>0 else 0.0
IWSR=("_iwsr_contrib", "mean")
```

Range & meaningful bands IWSR is bounded above by SR and below by FASR (a first-try win contributes the full 1.0, matching FASR; a 5th-try win contributes only 0.2). IWSR ≈ SR → fast repairs (Efficient Corrector). IWSR ≪ SR → slow, late convergence.

In the plots `iwsr_by_model_domain`, the IWSR bar in `sr_fasr_iwsr_by_model`, and the y-axis of `fasr_iwsr_scatter`.

Rationale The `1/k` weighting is the numerical encoding of Stechly's argument: if verification isn't easier than generation for a retrieval system, late successes deserve no credit for "search". IWSR is the

metric that turns the retry budget into a cost.

METRIC · RG: Retry Gap

Retry_Gap = Success_Rate - FASR How much of the reported success depends on retries. A pure diagnostic, not a capability — high is a warning, not a strength.

How it is computed A single subtraction on the `overall` / `by_domain` tables, then sorted ascending in the `retry_gap` table.

```
overall["Retry_Gap"] = overall["Success_Rate"] - overall["FASR"]
```

Range & meaningful bands RG¡0.10 → near-zero-shot. 0.10–0.30 → Efficient Corrector territory.¿0.30 → Stochastic Searcher: success is mostly resampling. In the bundled run qwen_3_5 hits 0.500.

In the plots `retry_gap_by_model` (diverging horizontal bars, red = positive gap). RG is also a ranked metric (direction = min) in the within-domain ranking.

Rationale RG operationalises the difference between "solved" and "solved on its own". It is the cleanest single read on whether a model's SR is trustworthy.

7.3.2 The iteration profile — the test that separates correction from resampling

RG tells you retries happened; the iteration profile tells you *whether they helped.* `compute_iteration_profile()` computes, for each attempt slot k, the conditional success probability among instances that needed exactly k attempts:

```
for k in range(1, max_iter+1):
    reached = df[df["Iterations"] >= k]
    exact   = reached[reached["Iterations"] == k]
    p_valid_given_reached = exact["Valid"].mean()
```

A **rising** curve means later attempts succeed more often — the model is using feedback (Efficient Corrector). A **flat** curve means success probability is independent of attempt number — each retry is an independent draw (Stochastic Searcher). `profile_is_flat()` formalises "flat" as max-min ≤ 0.10 over the finite points, and feeds the Stochastic-Searcher profile condition. This is the direct empirical test of Stechly's finding (iii) in your own data.

Reading caveat

With few instances per (model, k) slot the curve is noisy and `profile_is_flat` can return None (inconclusive) when fewer than two finite points exist. Treat the profile as suggestive, not decisive, on small task sets — the bundled run's 6-instance subsets are exactly this regime.

7.3.3 Reading the efficiency plots

The three success metrics are only diagnostic in combination. The guarantee `FASR ≤ IWSR ≤ SR` holds by construction, and the two gaps between them carry the meaning: the *SR–IWSR* gap measures how late successes arrive, the *IWSR–FASR* gap measures how much retries contributed at all.

FASR by model × domain. Two readings live in one chart.**Per model (compare a model's bars across domains):** a systematic gap between one domain group and another suggests memorised patterns specific to that group rather than generalised planning; a small gap across all domains points to genuine zero-shot transfer. A uniformly low FASR means near-zero zero-shot ability — go to SR-vs-FASR to check whether success is being manufactured by resampling.**Per domain (compare one domain across models):** a domain with consistently high FASR across every model is simply an easy domain — the domain, not the model, is driving the score.

SR vs FASR — the retry gap.

FASR by difficulty bin — genuine reasoning vs memorisation. An easy→hard sweep separates reasoning from training-frequency exploitation. A reasoner degrades *gradually and smoothly* as steps, objects and constraint interactions grow — the decline is a property of the task. A memoriser stays high on easy (short,

Gap pattern	Meaning
SR ≈ FASR, both high	Genuine zero-shot capability. Retries add almost nothing.
SR moderately¿FASR	Partial capability, partly scaffolded by the retry mechanism.
SR ≫ FASR (large gap)	Stochastic search. Per Stechly et al. (2023), retries are re-sampling, not correction — SR alone misleads.
Both near 0	No planning capability; the retry budget is irrelevant.

common-looking plans) and then falls off a *cliff* at the point where the required plans become long and rare, i.e. fall outside its training distribution. The cliff is the boundary of the distribution, not of a reasoning process.

FASR pattern (easy→hard)	Interpretation	Reference
Gradual decline	Generalisation — graceful degradation under complexity; some genuine reasoning that scales.	Expected for capable reasoners
Sharp cliff to ˜ 0	Memorisation — easy instances exploit training frequency; hard instances fall outside the distribution.	McCoy et al. 2024
Flat and high	Genuine capability across all difficulties. Strongest possible result.	Ideal case
Flat and low	No capability at any difficulty — or Difficulty='unknown' throughout (check data).	Valmeekam et al. 2023
Non-monotonic / irregular	Difficulty bins don't track real plan complexity. Recalibrate using optimal-plan-length quantiles / statistical rarity.	Data-quality issue

IWSR — where in the run the success happened. The closer IWSR sits to FASR, the more successes landed on attempt 1; the closer it sits to SR, the faster the model converges when it does eventually succeed. The two gaps read together:

SR–IWSR	IWSR–FASR	Interpretation	Profile
small	small	All three nearly equal; most success on attempt 1, retries add nothing.	Genuine zero-shot planner
small	large	IWSR ≈ SR ≫ FASR — rarely first-try, but converges quickly on attempt 2–3.	Efficient corrector
large	small	FASR ≈ IWSR ≪ SR — most success arrives very late, at attempt 4–5, and is heavily penalised by IWSR.	Stochastic / late luck
large	large	All three spread apart; SR is inflated by retries, while FASR reveals that true zero-shot performance is low.	Fully retry-dependent

P(Valid — reached attempt k). For each k the curve takes plans that actually *reached* attempt k (`Iterations ≥ k`) and reports the fraction resolving at exactly k — a conditional, not cumulative, probability. Its shape is the direct empirical test of the Stochastic-Searcher hypothesis.

Retry-gap bar & the FASR–IWSR scatter. The diverging retry-gap bar colours each model by `SR - FASR` (tomato for the usual positive gap; a steelblue/negative bar would be a data anomaly, since SR ≥ FASR by construction). Models at the top — lowest gap — are the most independent of the retry scaffold. The FASR-vs-IWSR scatter then encodes three axes at once:**x = FASR**(zero-shot skill),**y = IWSR**(retry efficiency),**dot size+colour = SR**(deep-red small = SR 0 → deep-green large = SR 1). No point can fall below the FASR=IWSR diagonal.

7.3.4 Execution and Reasoning

Executability ratio, Precondition Awareness Score, the sequencing-vs-fabrication split, and mean temporal distance — everything the PDDLSimulator extracts by replaying a plan step by step.

These metrics look *inside* a plan, valid or not, by replaying it through the `PDDLSimulator`. They are the framework's answer to "a plan failed — but how close was it, and what kind of mistake was itc". All are produced together by `compute_precondition_metrics`, which walks the plan step by step until the first precondition failure.

Line shape	Interpretation	Reference
Declining from k=1	Easiest problems resolve first; harder ones take more attempts — difficulty-sensitive correction.	Expected for a capable reasoner
Flat across k	Each retry is statistically independent — re-sampling, not correcting. Feedback content is irrelevant.	Stechly et al. 2023 (§4 Results)
Increasing with k	Model improves with each retry — genuine in-context learning. Very rare; check whether feedback leaks the answer.	Would contradict Stechly et al.
Low-flat near 0	No valid plans at any attempt; the iteration budget is wasted compute.	Valmeekam et al. 2023 (§5.2)

Position	Dot	Interpretation
Top-right, on/near diagonal	large green	Genuine planner — high FASR, IWSR≈FASR, high SR; success is first-attempt.
Top-left, well above diagonal	medium yellow	Efficient corrector — low FASR lifted by early-retry successes.
Bottom-left	small red	No capability — FASR and IWSR near zero, SR low.
Top-left, large dot, low FASR	large green/yellow	Dangerous case — high SR/IWSR mask retry dependence; SR misleads without FASR.

How one failure is classified

At the first failing action the simulator inspects each unsatisfied propositional precondition and asks `last_step_atom_true(atom)` against its per-step history `prop_hist`. If the atom was true at some earlier step → **sequencing error** (the model deleted a prerequisite too early). If it was never true → **state fabrication** (the model invented a state that never existed). A wrong-arity action is counted as fabrication and halts the walk.

METRIC · ER / Exec: Executability Ratio

exec_ratio = first_failure_index / plan_length How far into the plan the model gets before the first illegal action — a continuous measure of partial correctness that is completely independent of whether the goal is reached.

How it is computed The simulator applies effects step by step; at the first unsatisfiable precondition (or wrong arity) it records `first_failure`. A fully executable plan yields ratio 1.0 even if it never reaches the goal.

```
for step, action in enumerate(actions):
    ok, failed_prop, failed_num = sim.check_preconditions(...)
    if not ok:
        first_failure = step; break
    sim.apply_effects(...)
return {"executability_ratio": first_failure / plan_len}
```

Range & meaningful bands NaN for empty plans / missing PDDL. Bands:¡0.30 No-Grounding · 0.30–0.70 Vocabulary-Only ·¿0.70 Understander (runs deep, still fails the goal). In the bundled run kimi reaches 0.969 and gpt_oss 0.952 — both produce structurally near-perfect plans.

In the plots `executability_by_model_domain` (boxplots), `executability_vs_length` (faceted scatter — does executability decay with plan lengthc), and it is a radar axis and a ranked metric (direction max).

Rationale Huang 2022 established executability and goal-correctness as independent axes. A high exec with low SR is the Understander/Vocabulary signature: the model speaks the language and respects local preconditions but cannot sequence to the goal.

METRIC · PAS: Precondition Awareness Score

PAS = seq_errors / (seq_errors + fab_errors) Of the failures the model makes, what fraction are *ordering* mistakes (it understood the state model but mis-ordered) versus *fabrication*(it invented impossible states)c

How it is computed Computed at the single failure point. Each failed propositional precondition is bucketed via the temporal lookup; PAS is the sequencing share of all failures.

```
for atom in failed_prop:
    if sim.last_step_atom_true(atom) >= 0: seq_errors += 1     # was true before
    else:                                  fab_errors += 1     # never true
fab_errors += len(failed_num)
PAS = seq_errors / (seq_errors + fab_errors)   # NaN if no failures
```

Range & meaningful bands NaN when the plan fully executes (no failures) → substituted with a 0.5 prior so a perfect model is not penalised. High PAS + low SR = Understander; low PAS = Vocabulary-Only / incoherent world model.

In the plots The y-axis of `failure_mode_taxonomy`, a radar axis, and a ranked metric. The raw counts also drive `failure_type_breakdown`.

Rationale This is the metric that distinguishes Dziri's compositional collapse (sequencing) from Vafa's world-model incoherence (fabrication). Two models with identical SR and exec can sit at opposite ends of PAS and require completely different interventions.

7.3.5 Sequencing vs fabrication — the failure breakdown

`failure_breakdown` sums the two error counts per model and reports their proportions. In the bundled run almost every model is fabrication-dominant (proportion ≈ 1.0); the exception is gemma_4_31b at 0.42 sequencing / 0.58 fabrication — the only model in the set making a substantial share of "right idea, wrong order" mistakes rather than inventing states outright.

Model	seq	fab	seq prop.
gemma_4_31b	20	28	0.417
qwen_3_6_27b	5	19	0.208
nemotron(hf)	3	28	0.097
phi_4	1	32	0.030
gpt_oss_120b	0	1	0.000

Why strong models show tiny failure counts

gpt_oss and kimi show only 1 total failure each — not because they fail gracefully, but because they *succeed*
on most instances, so there is rarely a first-failure to classify. The failure breakdown is informative only where failures are plentiful; on near-solving models its proportions are statistically meaningless. Read it together with SR, never alone.

METRIC · τ: Mean Temporal Distance

τ = *mean(failure_step - last_true_step)* For sequencing errors only: how many steps back the violated prerequisite was last true. A measure of *how badly* the ordering collapsed.

How it is computed For each sequencing error the gap between the failure step and the last step the atom held is recorded; τ is the mean of those gaps.

```
last_step = sim.last_step_atom_true(atom)
temporal_distances.append(step - last_step)
mean_temporal_distance = np.mean(temporal_distances)
```

Range & meaningful bands Unbounded above; NaN when there are no sequencing errors (i.e. fabrication-only or fully-executing models — most of the bundled set). Small τ → the prerequisite was deleted just before it was needed (a local slip). Large τ → it was deleted many steps earlier (global ordering failure).

In the plots `temporal_distance_by_model` (per-model histograms). A ranked metric (direction min).

Rationale Refines PAS: among models that mostly make sequencing errors, τ says whether they are nearly-right (short τ) or have lost the plot (long τ) — the granular read on long-horizon compositional

ability.

7.3.6 Reading the execution plots together

These four figures are meant to be read as a cascade, not in isolation. `executability_ratio` is a prerequisite for PAS to mean anything: a model with exec near 0 has nothing to classify — it fails before any causal structure can be tested, so PAS is almost always NaN there. PAS becomes informative only once exec is moderate-to-high (≈0.4+), i.e. once the model has actually engaged the domain's state-transition logic long enough for its causal reasoning to be observable.

Executability ratio by model × domain (box plot). Box edges are Q1/Q3, the middle line the median, whiskers 1.5×IQR, dots beyond them outliers. A box near 1.0 means the model runs most of the plan before any failure (it understands domain structure); a box at 0.0–0.2 means it fails at step 1–2 (near-total incomprehension). A wide IQR means it works on some problems and collapses on others; a narrow IQR means consistency. The ideal is a **narrow box sitting high across every domain**— consistently good, most plans run before failing, little variation.

Cross-comparisons follow from the box meaning: comparing differently-coloured boxes *within* one domain cluster reads one model's behaviour across domains (its reasoning across scenarios); comparing the same colour *across* domains reads each domain's real difficulty.

Failure-type breakdown per model (stacked bar). Raw errors are summed across all domains/problems then converted to proportions (each row divided by its total), so models with different run counts stay comparable; a model with zero failures gets a flat zero bar rather than NaN.

Segment	Meaning	Severity
Blue — sequencing	The required atom was true earlier but the action was placed too late, after a prior effect deleted it. A tall blue bar = a reasonable causal model with ordering mistakes.	More fixable with better prompting
Red — fabrication	The required atom was never true in any prior state — the model invented a precondition. A tall red bar = no grounded world model.	Severe; less fixable by prompting alone

Atom vs precondition

A precondition is a conjunction of several
atoms
(single grounded propositional facts). The sequencing/fabrication verdict is made per failed atom at the first failing action, which is why a single action can contribute both kinds of error.

Executability ratio vs plan length (faceted scatter). The decay slope is the diagnostic. A negative correlation between plan length and exec_ratio is exactly Dziri et al.'s prediction (NeurIPS 2023): transformers lose causal coherence as the computation graph deepens. Faceting by domain separates a genuine state-tracking limit from a domain-specific one.

Pattern	Reading
Exec stays high regardless of length	Robust internal model.
Exec drops as length grows	State-tracking decay — the model leans on its own generated chain more than a strong internal representation. (A natural place for a Vafa-style Myhill-Nerode world-model metric, generalising PAS.)
Flat near 0 everywhere	Little-to-no domain knowledge, or the domain was never read.
Decays in *every* domain	Fundamental state-tracking capacity problem.
Decays in *one* domain only	Domain-specific problem.

Mean temporal distance per model (histogram). For a sequencing error, `distance = (step of first failure) -` averaged across the atoms of the failing precondition. Each bar groups plans whose sequencing errors had that mean distance. A model whose `mean_temporal_distance` is always NaN produces *no panel*— all its failures

are fabrications, there are no sequencing errors to measure, and the absence itself is informative: it means PAS = 0.0.

Distribution shape	Interpretation	Severity
Spike at 1–2	Near-miss ordering — almost right, one step too late.	Low
Spike at 3–7	Imprecise causal model — knows what's needed, tracks sequence loosely.	Moderate
Long tail 8+	Severe ordering collapse — preconditions were true much earlier.	High
No bars (NaN)	All failures are fabrications — no causal knowledge at all.	Severe

The cascade — read all four with one question chain

(1) Does the plan run at all (box plot)c (2) When it fails, is it conceptual or ordering (breakdown)c (3) Does the failure worsen with complexity (exec-vs-length)c (4) How severe are the ordering errors (temporal distance)c A model that passes (1), is mostly blue in (2), stays flat in (3) and spikes at distance 1 in (4) genuinely understands the domain and makes only minor sequencing slips. A model that fails (1) and has no histogram in (4) is fabricating preconditions from the first step — it may not be planning at all.

7.3.7 Grounding and Discriminatory

Action, fuzzy, and object hallucination, plus the inverse rate used downstream — the metrics that decide whether a model is reading the formal domain at all.

Hallucination is the most basic grounding signal: is the model even using the domain's legal vocabularyc A model that hallucinates is not reading the schema at all; a model that does not hallucinate has at minimum memorised it, even if it cannot sequence. Three variants separate three failure modes, all produced by `compute_hallucination_metrics` in one pass.

Why "discriminatory"

These metrics rarely correlate with success on their own — most capable models score 0 hallucination — but they are decisive at the
boundary
. They are what tells apart a Vocabulary-Only model (legal names, no coherent state) from a No-Grounding model (invents names outright). In the bundled run hallucination is 0 for almost every model; the exceptions are phi_4 (0.120) and qwen_3_6_27b (0.143).

METRIC · Halluc: Action Hallucination Rate

rate = illegal_action_names / total_actions Fraction of plan steps whose action name is not in the domain's legal action set. The headline grounding gate.

How it is computed Each parsed action name is checked for membership in `domain_info["action_names"]`.

```
if action_name not in legal_actions:
    hall_strict += 1
hallucination_rate = hall_strict / total_actions   # NaN if empty
```

Range & meaningful bands NaN for empty plans / absent domain file. The profile gate is strict: IHR¿0.90 (i.e. hallucination¡0.10) is required for the grounded profiles; IHR¡0.60 marks No Grounding.

In the plots `hallucination_heatmap` (model×domain), `hallucination_by_model_domain` (bars), the x-axis of `failure_mode_taxonomy`, and a ranked metric (direction min).

Rationale Valmeekam's obfuscation result (35%→0.16% when action names are scrambled) shows that apparent planning can collapse to nothing once the model can no longer pattern-match on familiar names. Hallucination rate is the everyday proxy for that grounding.

METRIC · Fuzzy: Fuzzy Hallucination Rate

counted only if min Levenshtein¿2 Strict hallucination minus the near-misses. An illegal name is counted here only if its edit distance to *every* legal name exceeds 2, so "pick-up" vs "pickup" is forgiven but a genuine invention is not.

How it is computed Among strict hallucinations, only those far from any legal name survive.

```
if min(levenshtein(action_name, a) for a in legal_actions) > MAX_FUZZY_DISTANCE:  # =2
    hall_fuzzy += 1
```

Range & meaningful bands Always ≤ strict rate. The *gap* between strict and fuzzy is the diagnostic: a model with high strict but low fuzzy is making transcription/format slips, not inventing actions — a parser/formatting problem, not a grounding problem.

In the plots Carried in the aggregate tables (`Fuzzy_Halluc`); read alongside the strict heatmap rather than plotted on its own.

Rationale Separates "the model doesn't know the action exists" from "the model knows it but spelled it differently". Only the former is a reasoning failure; the latter is fixable at the parser/prompt level and should not be charged against planning ability.

METRIC · ObjHalluc: Object Hallucination Rate

rate = illegal_arg_tokens / total_arg_tokens Fraction of argument tokens not present in the problem's `:objects` block — normalised over arguments, not actions. Catches models that know the verbs but invent the nouns.

How it is computed Every argument of every action is checked against `problem_info["objects"]` .

```
for arg in action_args:
    if legal_objects and arg not in legal_objects:
        obj_hall += 1
object_hallucination_rate = obj_hall / total_args
```

Range & meaningful bands An independent axis from action hallucination. High object-halluc + low action-halluc = the model has the action vocabulary but invents world entities — shallow grounding, exactly Vafa's incoherent-world-model signature at the object level.

In the plots `object_hallucination_by_model_domain` (bars).

Rationale Object grounding and action grounding fail independently. Splitting them prevents a model that uses legal actions on fabricated objects from looking grounded.

METRIC · IHR: Inverse Hallucination Rate

IHR = 1 - hallucination_rate The same signal flipped so higher is better, for use in the composite score and the profile thresholds.

How it is computed `1 - hallucination_rate` , NaN-preserving.

```
inverse_hallucination_rate = 1 - hallucination_rate
```

Range & meaningful bands IHR is the radar axis and composite component (weight 0.20 via `one_minus_halluc`). Profile gates:¿0.90 grounded ·¡0.60 No Grounding.

In the plots Radar axis (`radar_chart`); component of the Composite Planning Score.

Rationale Purely a sign convention so every composite input points the same direction (higher = better) and the weighted sum stays interpretable.

NaN handling differs by consumer

In the aggregate tables hallucination is averaged NaN-aware. But inside the per-row bootstrap for the composite CI, a NaN hallucination is replaced by a neutral 0.5 for `one_minus_halluc` rather than dropped. This is deliberate (an empty plan shouldn't score as perfectly grounded) but it means the composite CI and the table mean can diverge for models with many empty plans — see section 06.

7.3.8 CoT Alignment (CoTaL)

From a capped text-overlap proxy to a five-component structural comparison of the emitted plan against the plan reconstructed from the reasoning trace — plus the status taxonomy that says how far to trust each score.

CoTaL — the CoT plan-alignment score — answers the question Lanham 2023 and Kambhampati 2024 pose: *does the model's reasoning actually correspond to what it does, or is the chain-of-thought a post-hoc story?* The metric began as a crude text-overlap proxy and was redesigned into a structural sequence comparison between two reconstructed plans. Both still exist in the code, but they play different roles: the structural score is the real metric; the text proxy is a capped fallback for when there is only one plan to look at.

The two-plan idea

For a reasoning model, the parser produces **two** plans per attempt: `raw.actions` (what the model emitted in its answer) and `reasoning.actions` (the plan reconstructed from the thinking trace). CoTaL is, at its strongest, the structural agreement between these two sequences. High agreement = the model did what it reasoned about (faithful CoT). Low agreement = the answer and the reasoning diverged.

7.3.9 The status decision tree

`compute_cot_alignment_for_attempt` routes each attempt into one of several statuses depending on which plans exist and whether they validate. The status is as important as the number — it tells you how much to trust the score.

Status	Condition	Score basis	Confidence
comparable_and_both_valid	raw + reasoning plans, both valid	structural_alignment	high
comparable_but_both_invalid	both plans present, both invalid	structural_alignment	high
comparable_but_raw/reasoning_invalid	both present, one invalid	structural_alignment	high
semantic_proxy_only	raw plan but no reasoning plan	text overlap × 0.35 cap	low
no_raw_plan	reasoning but no emitted plan	none	low
no_reasoning_text	neither reasoning text nor actions	none	none

7.3.10 The strict metric — structural_alignment

When both plans exist, `compute_sequence_alignment` (in `evaluators/plan_alignment.py`) compares the two normalised action sequences along five order- and content-sensitive dimensions and combines them into one bounded score:

$$\begin{aligned}\text{structural alignment} \in [0,1] = {} & 0.35 \cdot \text{lcs\_ratio} + 0.25 \cdot \text{prefix\_ratio} \\ & + 0.20 \cdot \text{bag\_overlap} + 0.10 \cdot \text{length\_ratio} + 0.10 \cdot \text{repetition\_similarity}\end{aligned}$$

Alongside the score it returns rich diagnostics the report can drill into: `exact_sequence_match`, `first_mismatch_index`, `ad` localisation), `displaced_actions`, per-sequence distinct-action counts, action frequencies, and run-length `repetition_bloc` These are exactly the distinct/repetition/transposition diagnostics the redesign called for.

7.3.11 The fallback — semantic proxy, capped at 0.35

When only the raw plan exists (no reconstructable reasoning plan), the metric falls back to `compute_cot_semantic_support` the fraction of the plan's action names and object names that appear as tokens in the reasoning text. This is vocabulary coverage, not plan agreement, so it is deliberately **capped at 0.35**— it can never masquerade as a strong structural match.

```
score = semantic["cot_semantic_support_score"]          # (action_cov + object_cov)/2
proxy_score = score * semantic_proxy_cap                  # cap = 0.35
status = "semantic_proxy_only"; confidence = "low"
```

Component	What it captures	Why weighted so
lcs_ratio	longest common subsequence / max len — order-preserving overlap	highest (0.35): the truest measure of "same plan in the same order"
prefix_ratio	common leading actions / min len — do they start identically?	0.25: a shared prefix is strong evidence of a shared plan
bag_overlap	multiset intersection — same actions, order ignored	0.20: same ingredients even if reordered
length_ratio	min/max length — are the plans the same size?	0.10: weak signal, easily coincidental
repetition_similarity	similar amount of repeated actions	0.10: catches degenerate looping

Why the cap matters

Mentioning the right words is not the same as planning them in the right order. The cap encodes that epistemic gap: a proxy score of 0.35 is the ceiling of "the reasoning at least talks about the plan", which is strictly weaker than "the reasoning *is* the plan".

7.3.12 Which plan is the basis, and which attempt is selected

When both plans validate-differently, a three-way rule picks the authoritative one: prefer the valid plan if only one is valid; if both are valid prefer the *shorter*; if both invalid prefer the reasoning plan. Across iterations, `select_cot_alignment_attempt` takes the first solved attempt when the instance is solved, otherwise the last attempt — so the reported CoTaL reflects the decisive attempt, not an average of doomed ones.

```
if   raw_valid and not reasoning_valid: basis = "raw"
elif reasoning_valid and not raw_valid: basis = "reasoning"
elif raw_valid and reasoning_valid:     basis = "raw" if len(raw) <= len(reasoning) else "reasoning"
else:                                   basis = "reasoning"
```

7.3.13 Aggregation & range

The `overall` table actually exposes three CoT aggregates: `strict_mean_cot_plan_alignment` (structural only), `inclusive_mean_cot_alignment` (strict-or-proxy), and `mean_cot_semantic_support` (raw coverage), plus status-rate columns like `cot_comparable_and_both_valid_rate` and `cot_exact_sequence_match_rate`. Report the strict mean as the metric and the status rates as the coverage context.

The single most important caveat for this metric

CoTaL is only defined where paired plans (or at least reasoning text) exist. In the bundled run the status distribution is **282 no_reasoning_text · 82 semantic_proxy_only · 18 comparable_and_both_valid** — most instances have *no* reasoning plan to compare, so CoT_Alignment is NaN for most non-reasoning models. Where it is defined on tiny samples it can be misleading: kimi shows CoT_Alignment = 1.000 driven by a handful of comparable attempts while its SR is only 0.051. Never read a CoTaL mean without its sample size and status mix; a high CoTaL on a low-SR model is the textbook "the reasoning matches a plan that still fails" case, not evidence of capability.

Known accessor edge case

`parsed_plan_reasoning_text()` returns the reasoning field only when it is a string; for the current dict-shaped

`reasoning`
namespace it returns "". In practice the reasoning text is recovered first from the raw generation via `raw_attempt_reasoning_text()`
, so the proxy path still fires — but if a run lacks raw generation text, the proxy will silently see an empty string. Worth gating with an explicit diagnostic.

7.3.14 The coverage proxy: the double-intersection

Underneath the structural metric sits the semantic-support proxy — the layer that answers the plain question *"does the chain-of-thought actually talk about the same things that end up in the plan?"* If the CoT names the right actions and objects, that is evidence the reasoning guided generation (high score); if it talks about entirely different things, the reasoning is decorative and was likely written around a plan produced by some other mechanism (low score). This operationalises Kambhampati's "fake reasoning vs genuine deliberation" and Lanham et al.'s finding that CoT grows less faithful with scale.

Coverage is computed with a deliberate **two-step filter**:

Both filters are necessary.**Without the first**, any English word resembling a PDDL action name would inflate the score; restricting to exact members of `legal_action_names` stops that.**Without the second**, a CoT that simply lists every action in the domain would score 1.0 even if the plan used one action; requiring the specifically-used actions stops that.

Why the denominator is the plan, not the CoT

Each coverage score divides by the size of the
plan
vocabulary, not the CoT vocabulary. The question is "what fraction of what the model
did
is reflected in its reasoning?", not "what fraction of the reasoning made it into the plan?". A plan-based denominator rewards coverage of what was actually done and denies a long, everything-mentioning CoT an inflated score.

Worked example. CoT:*"move car1 from l1 to l2 along the road; once car1 is at l2, pick up passenger p1; then drive to l3 to drop off the passenger."* Plan: `(move-car car1 l1 l2) (pick-up p1 car1 l2) (move-car car1 l2 l3) (dr`
Every plan action and object is named in the prose, so both action- and object-coverage are full.

Scenario	action_cov	object_cov	What it reveals
CoT names every action and object in the plan	1.00	1.00	Perfect alignment — the CoT anticipated the plan.
All actions, wrong objects	1.00	0.00	Knows what to do, reasons about the wrong objects.
Some actions, all objects	0.50	1.00	Object tracking good, action selection partly off.
No domain actions or objects	0.00	0.00	CoT is pure prose with no PDDL grounding.
Empty plan / empty CoT	0.00	0.00	max(0,1) denominator avoids division by zero → 0.

Tokeniser is substring-based, not word-boundary

Tokenisation uses
`re.findall(r'[a-z][a-z0-9_-]*', cot_lower)`
, which matches substrings rather than whole words. If the CoT contains "pick-up-person" and the action is "pick-up", the shorter token still matches, so the score can be slightly inflated when an action name is a substring of a longer compound word. Rare for specific PDDL action names, but worth remembering when reading very high scores.

7.3.15 Reading the CoT plots

Mean CoT alignment by model × domain (bars). The mean hides variance, so it is the first thing to look at before the violin. A mean of 0.60 could be every plan near 0.60, or half at 1.0 and half at 0.2 — the violin resolves it; read both. Across a model's domains: similar-and-high means domain-general grounding (formal

vocabulary used regardless of domain); dissimilar means domain-specific behaviour (grounded in some domains, lapsing into prose in others — a transfer failure); similar-and-low means the CoT is not grounded in PDDL vocabulary at all. Comparing one domain across models yields a qualitative sense of how hard that domain's names/identifiers are for CoT to latch onto.

Pattern	Conclusion
CoT=True > False	CoT is causally helping —*but* only counts as genuine reasoning if the alignment score is also high; if success rises while alignment is low, the CoT is not what improved the outcome (Stechly et al. 2023).
CoT=True < False	CoT is actively harmful — verbose self-contradicting reasoning confuses plan generation.
CoT=True ≈ False	CoT is decorative — Kambhampati's "fake reasoning": plans retrieved by pattern-matching, CoT written around them.
Both low	Model cannot plan here regardless; the CoT effect size is not interpretable at the floor.

Success rate: CoT=True vs CoT=False (paired bars).

This contrast is only valid if everything else is held equal

The CoT=True/False runs must test the same problems under the same prompt, **temperature** and **iteration budget**, or the bar difference reflects confounds rather than a CoT effect. Temperature divides the logits before sampling (adjusted = score/temperature): < 1 sharpens toward the most likely tokens, > 1 flattens toward surprising ones — so a reproducible benchmark should pin temperature to 0 or near-0 for every model so differences reflect capability, not sampling luck. The iteration budget (N rounds) follows Valmeekam et al. 2023 for the cap and Stechly et al. 2023 for treating attempts past the first as independent samples; 3–15 is the defensible range since the marginal value of later iterations is negligible. Both must be identical across the True and False runs.

Alignment distribution for valid vs invalid plans (violin). Compare the median lines between the Valid and Invalid violins: a substantially higher Valid median means alignment is predictive of success (models reasoning in domain vocabulary are likelier to succeed); equal medians mean alignment and validity are independent. The violin is a 90°-rotated, KDE-smoothed histogram (x = alignment score, y = plan count) — and because CoTaL scores cluster at the extremes, the KDE is imperfect and should be read loosely.

Valid violin	Invalid violin	Conclusion
Tight, high mass near 1.0	Wide, spread 0–1	Valid plans have consistently aligned CoT; failed plans have chaotic CoT — no reasoning↔output relationship.
Moderately tight, high	Slightly lower, similar width	CoT stays grounded even in failures — failures are sequencing/complexity, not reasoning misalignment.
Very wide, mid-range	Wide, similar	CoT noisy for both outcomes — alignment is essentially random.

7.3.16 PS (Composite Planning Score)

What PS represents, its five weighted components plus the conditional CoT bonus, the per-row bootstrap CI, how to read the capability radar, and how to retune the weights for a specific research goal.

The Composite Planning Score collapses five orthogonal capability axes into one number per model (and per model×domain). Its purpose is triage, not verdict: a single ranking to scan, always reported alongside its components because a composite can hide compensating strengths and weaknesses.

7.3.17 Why a composite at all

The preceding sections give nine-plus dials. PS exists so that, for a quick comparison, you have one axis that already encodes the framework's value judgement: zero-shot ability matters most, retries are a cost, structural quality and grounding matter, and precondition awareness is a tie-breaker. It is the metric you sort the leaderboard by; it is never the metric you conclude from.

7.3.18 The components and their weights

PS is a weighted sum of five inputs, each already in [0,1], with an optional sixth (CoT) bonus. The person sometimes thinks of it as "two components" — that intuition maps onto the structure exactly: there is a **base**(the five-term weighted sum) and a **conditional CoT bonus** that only applies when a strict CoT alignment score exists, at which point all weights are renormalised so the result stays bounded.

$$\text{base} = 0.25 \cdot \text{FASR} + 0.20 \cdot \text{IWSR} + 0.20 \cdot \text{exec\_ratio} + 0.20 \cdot (1 - \text{halluc}) + 0.15 \cdot \text{PAS}$$

$$\text{PS} = \frac{\text{base} + 0.05 \cdot \text{CoT}}{1.05}, \quad \text{if CoT is finite}$$

$$\rightarrow \text{renormalised and clipped to } [0, 1]$$

Component	w	Rationale for the weight	NaN substitute
FASR	0.25	Highest — the only signal that cannot be inflated by retries	0
IWSR	0.20	Retry efficiency; rewards fast convergence, penalises stochastic search	0
exec_ratio	0.20	Structural quality independent of goal success	0
1 - halluc	0.20	Schema grounding	0.5
PAS	0.15	Lowest — undefined for zero-failure models, so a 0.5 prior is needed	0.5
CoT bonus	+0.05	Applied only when strict CoT alignment is finite	skipped

7.3.19 Aggregation & range

PS is computed on the *aggregated* components in both the `overall` and `by_domain` tables — so each model carries an overall PS and one PS per domain. In the bundled run: gpt_oss 0.548 · nemotron(nv) 0.446 · kimi 0.435 · qwen_3_5 0.425, down to phi_4_mini 0.315.

7.3.20 The 95% bootstrap confidence interval

To show whether a PS difference is real or noise, the `composite_score` table attaches a bootstrap CI (N=1000, seed=42). Crucially it resamples **per-row PS contributions**, not per-model means — a more conservative estimate that reflects instance-level variability.

```
contribs = model_df.apply(per_row_composite, axis=1).values
boots = [contribs[rng.integers(0,len(contribs),len(contribs))].mean() for _ in range(1000)]
PS_ci_lo, PS_ci_hi = np.percentile(boots,2.5), np.percentile(boots,97.5)
```

Overall PS and the CI are computed two different ways

The overall PS averages the components first, then weights them. The CI averages per-row composites. These are not the same operation — for non-linear cases and for models with many empty/NaN rows they can diverge, and the overall PS can even sit outside its own CI (in the bundled run kimi's PS_overall 0.435 lies above its CI [0.184, 0.247]). Read the CI as a stability indicator for the per-instance score, not as a strict interval around PS_overall. If you need them to coincide, compute the point estimate the same per-row way.

7.3.21 The capability radar

The `radar_chart` plots the five [0,1] axes **FASR · IWSR · Exec · IHR · PAS** as a pentagon per model. It is the visual companion to the profile thresholds: a Genuine Planner fills most of the pentagon; a No-Grounding model collapses to the centre; a Vocabulary-Only model shows a lopsided shape — high IHR but low PAS and Exec. Read *area* for overall strength and *shape* for the capability signature. Note the radar uses IHR (not 1-halluc) and omits the CoT bonus, so it is the *base* profile, not the bonus-adjusted PS.

Radar reading caveat

Equal area can come from very different shapes — a model strong on FASR/IWSR but weak on Exec/PAS can match the area of a balanced model. Always compare shape, not just fill. With many models the overlapping fills become unreadable; facet or pick a handful for any figure you publish.

7.3.22 Goal-dependent weight tuning

The weights encode a scientific claim; changing them changes the question PS answers. They live in one dict (`COMPOSITE_WEIGHTS`) and must sum to 1.0. Reasonable retunings for different research goals:

Research goal	Adjustment
Zero-shot capability only	FASR→0.40, IWSR→0.10
Domain grounding focus	one_minus_halluc→0.35, FASR→0.15
All dimensions equal	all five = 0.20 (simple average)
Execution-centric	exec_ratio→0.35, PAS→0.10

The discipline that makes PS safe

Always report the component scores next to PS. A PS of 0.60 from FASR=1.0 / exec=0.0 (always right on the first try, never produces a partially-executable failure) is a completely different model from a PS of 0.60 from moderate scores everywhere. The composite ranks; the components explain.

7.4 Cross-Domain and Cross-LLM comparison

Within-domain ranking and its direction rules, the per-domain heatmap normalisation caveat, rank variance (generalist vs specialist), Spearman domain correlation, the failure-mode taxonomy quadrants, and the seven literature-grounded capability profiles with their NaN-aware assignment logic.

A single overall leaderboard hides the most scientifically interesting fact about a model:*where* it is good. This view documents the three machinery pieces that turn per-domain metrics into comparative claims — within-domain ranking, rank variance, and the Spearman domain-correlation matrix — and then the failure-mode taxonomy and the seven literature-grounded capability profiles that convert those numbers into a named diagnosis.

7.4.1 Within-domain ranking: why rank, not raw value

Absolute metric values are not comparable across domains — a 0.30 success rate in a hard transport domain and a 0.30 in an easy gripper domain mean different things. The framework therefore ranks models **inside each domain separately**, on every metric in `RANK_METRICS`, then works with ranks. Ranking removes the domain-difficulty offset and answers the only question that survives it:*among the models that faced this exact domain, who placed where?*

The direction of "good" is metric-specific and is encoded once in `RANK_METRICS`. Maximise for Success_Rate, FASR, IWSR, Exec, PAS, CoT_Alignment; minimise for Hallucination, Retry_Gap, Temporal_Distance. The rank call makes the direction explicit:

```
for metric, direction in RANK_METRICS.items():
    rank_table[metric] = sub[metric].rank(
        ascending = direction == "min",   # min-metrics: smallest value = rank 1
        method    = "min",                # ties share the best rank
        na_option = "bottom",             # missing values rank last, never first
    )
# rank 1 = best in this domain on this metric
```

The rationale of the procedure

Ranking is a deliberate refusal to average raw scores across heterogeneous domains. "Model A beats Model B" is only ever asserted
within a domain on a named metric
; a global statement is then built by aggregating those local verdicts (mean rank, rank variance), never by averaging raw metric values that live on incomparable scales.
`na_option="bottom"`

is a fairness rule: a model that produced no usable plan cannot be handed a good rank by accident.

7.4.2 The ranking heatmap and its hard caveat

The `domain_ranking_heatmap` renders one sub-grid per domain (rows = models, columns = RANK_METRICS). To share one colour scale across domains with different model counts, each raw rank is min-max normalised inside its domain to [0,1]:

$$\text{norm\_rank} = \frac{\text{rank} - 1}{\max(\text{n\_models} - 1, 1)}$$
$$\in [0, 1], \quad 0 = \text{best in domain}, \quad 1 = \text{worst in domain}$$
$$\text{Colormap: RdYlGn\_r, green} = \text{best.}$$

Do not read normalised cells as cross-domain quality

The normalisation is **per-domain**. A 0.00 (green) cell means "best *among the models present in this domain*", not "good in absolute terms". A model can be deep green in a domain where every model failed. Compare cells *down a column within one domain block*; never compare the same model's colour across two different domain blocks as if it were a score. The colour encodes local standing, not capability.

7.4.3 Rank variance: generalist vs specialist

Rank variance compresses "where is this model good" into one number. For each model it takes the standard deviation of its within-domain rank across domains, per metric, then averages those across metrics (`mean_rank_std`). A model that places ~ 3rd everywhere has near-zero variance (a generalist); a model that wins one domain and trails in another has high variance (a domain specialist).

```
var_df = rank_within_domain.groupby("Model")[RANK_METRICS_cols].std()
var_df["mean_rank_std"] = var_df[RANK_METRICS_cols].mean(axis=1)
var_df = var_df.sort_values("mean_rank_std", ascending=False)  # specialists on top
```

phi_4 carries the highest `mean_rank_std` $\approx$ `2.394` (most domain-specific — strong in a domain or two, weak elsewhere) while gpt_oss sits lowest at $\approx$ `1.349` (most consistent across domains). High variance is not "bad": it flags a model whose single overall number is least trustworthy, because its behaviour is domain-contingent.

Spec says "normalised rank", code uses raw rank

The plot docstring describes rank variance as the std of *normalised* rank, but the implementation computes std over the *raw* integer ranks in `rank_within_domain` (the normalisation is applied only inside the heatmap). With a fixed model count the two differ by a constant factor, so the ordering of models is unaffected — but the absolute `mean_rank_std` values are on the raw-rank scale, not [0,1]. Worth aligning if you ever compare variance magnitudes across runs with different model counts.

7.4.4 Domain correlation: redundancy vs complementarity

The `domain_correlation` plot is a model×model Spearman ρ matrix built from each model's vector of per-domain success rates. Two models with $\rho \approx 1$ succeed and fail on the same domains — they are *redundant* probes of capability. Two with low or negative ρ stress different things — they are *complementary*, and a benchmark wants both. It is a tool for curating the model set and the domain set, not for ranking.

```
sr_pivot = df_metrics.pivot_table(index="Model", columns="Domain", values="Success_Rate")
corr = sr_pivot.T.corr(method="spearman")   # Model \(\times\) Model rank correlation
```

Correlation needs a domain spread

Spearman over success-rate vectors is only meaningful with several domains and some variance in them. In the bundled run the domain count is small and many models sit at SR≈0, so ρ is dominated by ties and should be read as illustrative of the method, not as a stable finding.

7.4.5 Breadth of competence: stacked PS and the SR heatmap

`ps_by_domain_stacked` stacks each model's per-domain Planning Score contributions into one bar. Two models can reach the same total bar height by opposite routes: a tall single segment (high PS concentrated in one domain) versus many even segments (competence spread across domains). The stack makes "broad vs concentrated" visible at a glance — exactly the distinction the overall PS scalar destroys.

The `success_rate_heatmap` is the raw companion: model×domain SR with no normalisation, so here — unlike the rank heatmap — cells *are* comparable across the whole grid. Use it to spot empty columns (domains no model solved → an infrastructure or difficulty problem, not a model problem) and empty rows (models that solved nothing).

7.4.6 The failure-mode taxonomy plot

This is the framework's single most diagnostic figure. It places every model on two orthogonal failure axes —**hallucination rate**(x, grounding failure) against **PAS**(y, precondition awareness) — encoding **FASR as bubble size** and **SR as colour**. The point is that "the model failed" is not one phenomenon: the two axes separate four qualitatively different failure regimes.

Quadrant	Reading	Diagnosis
low halluc · high PAS	grounded, legal names, but mis-orders actions	Sequencing failure (e.g. gemma)
low halluc · low PAS	legal names, no coherent state model	Vocabulary-only
high halluc · high PAS	invents actions yet shows some precondition sense	Confused / mixed
high halluc · low PAS	not reading the domain; everything ungrounded	No grounding

What the taxonomy lets you conclude

Two models with identical success rates can sit in different quadrants — and that difference is the actual research finding. In the bundled run gemma is the only sequencing-heavy model (failure breakdown 20 sequencing vs 28 fabrication, seq-proportion 0.417), so it lives in the grounded/high-PAS corner while the flat-zero models collapse toward the ungrounded corner. The taxonomy turns an undifferentiated pile of failures into named, citable failure phases.

7.4.7 The seven capability profiles

The profiles are the taxonomy made formal: each is a conjunction of threshold conditions over `{SR, FASR, IWSR, RG, IHR,` and each maps to a specific claim in the planning literature. They are the bridge from "here are nine numbers" to "this model is doing X".

7.4.8 How a model is assigned a profile

Assignment is deliberately conservative. Every profile is evaluated; the **first exact match**(all required conditions met, none unavailable) wins. If nothing matches exactly the model is labelled `Mixed/Unclassified` and reported against its *closest* profile — the one with the highest matched-condition count, ties broken by fewest required conditions.

```
evaluations = [evaluate_profile_conditions(metrics, p) for p in PROFILE_DEFINITIONS]
exact = next((e for e in evaluations if e["exact"]), None)
if exact is None:
    best = max(evaluations, key=lambda e: (e["match_count"], -e["required_count"]))
```

Profile	Conditions	Interpretation · reference
Genuine Planner	SR¿0.70 · FASR¿0.50 · RG¡0.10 · IHR¿0.90 · CoT¿0.70	Reads the schema, solves first try, reasoning tracks the plan.
Strongest claim		
Efficient Corrector	SR¿0.50 · FASR¡0.30 · —SR-IWSR—≤0.10 · 0.10≤RG≤0.30	Rarely first-try, but repairs land early so IWSR stays near SR.
Stechly 2023 (verify via P(Valid—k))		
Stochastic Searcher	SR≥0.30 · FASR¡0.20 · RG¿0.30 · [opt] P(Valid—k) flat	Success inflated by retries; attempts behave like resampling.
Stechly 2023		
Lucky Retriever	SR¿0.60 · FASR¿0.50 · IHR¡0.30 · CoT¡0.40	Solves from surface patterns; weak grounding, post-hoc reasoning.
Kambhampati 2024 · Valmeekam obfuscation		
Understander	SR¡0.20 · FASR¡0.10 · ER¿0.70 · IHR¿0.90 · PAS¿0.70	Knows vocabulary &local preconditions, fails long-horizon order.
Valmeekam Phase 2 · Dziri 2023		
Vocabulary-Only	SR¡0.15 · IHR¿0.90 · 0.30≤ER≤0.70 · PAS¡0.30	Legal action names, no coherent state-transition model.
Kambhampati 2024 · Vafa 2024		
No Grounding	SR≤0.05 · IHR¡0.60 · ER¡0.30 · PAS¡0.30	Not reading the formal domain at all; everything ungrounded.
Valmeekam 2023 · McCoy 2024		

```
    assigned = "Mixed/Unclassified"      # honest about partial matches
else:
    assigned = exact["profile"]
```

NaN-aware conditions: "unavailable" ≠ "missing"

A condition whose metric is NaN (e.g. CoT when reasoning was never produced) is marked *unavailable*, not *failed*. This is what stops a model being thrown out of "Genuine Planner" merely because CoT could not be computed — and it is why the report exposes the full matched/missing/unavailable breakdown per profile, so you can see exactly how close a model sat to each boundary rather than trusting a bare label.

Profiles are thresholds, not ground truth

The cut-points (SR ¿ 0.70, IHR ¿ 0.90, ...) encode the literature's qualitative claims as hard numbers; they are defensible but not sacred. A model just under a boundary is reported as Mixed against its closest profile precisely so the threshold choice stays visible and auditable. Treat the label as a hypothesis the condition breakdown lets you check,

not a verdict.

7.4.9 Reading the metric set together: the diagnostic framework

No single metric is a verdict. The scientific meaning lives in *combinations*— and the within-domain ranking heatmap is where they are read side by side. There are three reading directions, and then a small set of metric pairs that must never be read in isolation.

Direction	What it tells you
A **row**(one model)	Scan all metrics: uniformly green = leads on every dimension here; uniformly red = worst on everything; a mixed row is a capability profile — strengths and weaknesses at once.
A **column**(one metric)	Which model dominates that metric. A column all one colour means the metric doesn't discriminate in this domain.
Cell blocks	Green SR+FASR but red halluc+CoT = succeeding despite poor grounding (pattern-matching). Red SR but green exec+PAS = failing at the plan level yet showing process-level understanding.

Three ways to read the ranking heatmap.

The metric pairs to always read together. Capability triangle — SR + FASR + IWSR + RG(is performance genuine or scaffolded by retriesc). Read SR and FASR first, then RG = SR - FASR, then IWSR position.

SR	FASR	RG	IWSR	Diagnosis
High	High	≈0–0.10	≈FASR	Genuine zero-shot planner — retries irrelevant.
High	Medium	0.10–0.25	≈SR	Efficient corrector — converges on attempts 2–3.
High	Low	¿0.30	≈FASR	Stochastic search — SR inflated; late retries are re-samples.
Medium	Low	0.20–0.40	mid	Partial capability, retry-scaffolded.
Low	Low	≈0–0.05	≈FASR	No capability — nothing for retries to inflate.

Failure anatomy — ER + PAS(most useful when SR¡0.15: how deep does it run, and why did it stopc)

ER	PAS	What the model is doing	Valmeekam phase
¿0.70	¿0.70	Runs deep, ordering errors near the goal — understands domain, mis-sequences late.	Phase 2 (long-horizon)
¿0.70	¡0.30	Runs deep but fabricates states — knows vocabulary, invents transitions.	Phase 1/2 boundary
¡0.30	¿0.70	Fails early, but on ordering of the few real steps. Rare.	Phase 2 + vocab issues
¡0.30	¡0.30	Fails immediately, invents states from step 1.	Phase 1 (vocabulary)
Medium	NaN	Mixed; PAS undefined — check halluc gate and temporal distance.	Mixed

Reasoning test — SR + CoT and **refinement pair — IHR + PAS**(genuine vs retrieved success; and which kind of failure).

IHR is a gate, not just a metric

If inverse-hallucination rate falls below ˜ 0.6, ER, PAS and CoTaL are all unreliable: the model is failing at vocabulary, so the downstream causal metrics have nothing trustworthy to measure. Always clear the IHR gate before interpreting the process metrics.

The diagnostic tree. The pairs above compose into a single ordered procedure — read the metrics in this sequence, taking the branch that matches, and the model classifies itself.

7.5 Graph Catalogue

Every figure the pipeline emits, documented by scope, reading procedure, the data-availability problems that distort it on sparse runs, and the justification for its chart type — grouped by metric family.

Pair	Signature	Meaning
SR high · CoT¿0.70	—	Reasoning guides grounded plans — strongest evidence of planning (Lanham).
SR high · CoT¡0.40	—	Fake reasoning — correct plans, post-hoc CoT (Kambhampati).
SR low · CoT¿0.70	—	Faithful but failing — reasoning grounded, execution fails.
IHR¿0.90 · PAS¿0.70	top-left (good)	Knows vocabulary, ordering errors only — most fixable.
IHR¿0.90 · PAS¡0.30	bottom-left	Knows words, lacks world model.
IHR¡0.60 · PAS low	bottom-right (worst)	Near-random output — no grounding, no causal model.

IHR¡0.60	STOP. Not reading the schema — vocabulary failure. Do not interpret exec/PAS/CoT.
IHR 0.60–0.90	Proceed with caution; note partial vocabulary issues.
IHR¿0.90	Proceed to Level 1.

The pipeline emits up to 27 figures, registered by name in `PLOT_DESCRIPTIONS` and written by `maybe_make_plots`. This catalogue documents each one along the four axes that decide whether a figure earns its place: its **scope**(what slice of data it shows),**how to read it**, the **data-availability problems** that distort it on sparse runs, and the **justification** for that chart type over the alternatives.

One reading rule that governs half the figures

Several plots are bounded [0,1] and zero-inflated: on weak models the bars genuinely collapse to zero, and a zero can mean either "good" (no hallucination) or "nothing produced" (no actions emitted). Whenever a bar is flat-zero, disambiguate against Success_Rate and plan counts before reading it as a strength. The atlas notes this per-plot where it bites.

7.5.1 Grounding / discriminatory

Hallucination heatmap

`hallucination_heatmap`
chart heatmap
scope Mean action-hallucination rate per model×domain. Lower = better grounded.
how to read Read down a column to compare models in one domain; dark = invents illegal actions. Empty cells = no plans produced there.
data problems A model with SR≈0 can still show 0 hallucination if it never emitted a parseable action — read alongside Exec.
why this form Heatmap is right because the data is a dense model×domain grid of one bounded scalar; small multiples would waste space.

Action hallucination (bars)

`hallucination_by_model_domain`
chart grouped bars
scope Same action-hallucination rate, grouped bars per model across domains.
how to read Bar height = fraction of emitted actions whose name is not in the domain schema.
data problems Zero-height bars are ambiguous: genuinely grounded vs nothing emitted. Cross-check plan counts.
why this form Bars expose per-domain spread for one model better than a heatmap row when domain count is small.

SR¡0.15	Almost no valid plans — skip to Level 4; the question is which failure dominates.
SR 0.15–0.50	Partial capability — Level 2.
SR¿0.50	Can plan here — Level 2 to test authenticity.

RG¡0.10	Retry-independent (FASR≈SR) — Level 3 + Level 5.
RG 0.10–0.30	Moderate dependency — report SR and FASR; Level 3.
RG¿0.30	Strong dependency — SR inflated, use FASR; confirm via flat P(Valid—k).

Object hallucination

`object_hallucination_by_model_domain`
chart grouped bars
scope Fraction of action arguments referencing objects absent from the problem file.
how to read Separates' wrong object' errors from' wrong action' errors; compare against the action-hallucination twin.
data problems Undefined when a model emits no arguments; appears as 0 and must not be read as' good'.
why this form Splitting object- from action-hallucination is the whole point — two distinct grounding failures need two plots.

7.5.2 Execution & reasoning

Executability ratio

`executability_by_model_domain`
chart box / strip
scope Distribution of exec_ratio (valid-prefix length / plan length) per model×domain.
how to read Higher = plans run further before the first precondition break. 1.0 = fully executable (often also valid).
data problems Single-instance domains give a degenerate' distribution' of one point; spread is only meaningful with several instances.
why this form A distribution view (not a mean) is justified because exec_ratio varies instance-to-instance and the spread is informative.

Executability vs plan length

`executability_vs_length`
chart scatter (faceted)
scope exec_ratio against plan length, one facet per domain.
how to read A downward trend = the model breaks down on longer horizons (the Embers-of-Autoregression length cliff).
data problems Needs many instances spanning a length range; sparse runs show a near-empty scatter with no visible trend.
why this form Scatter is the only honest way to show a per-instance relationship between two continuous quantities.

Failure type breakdown

`failure_type_breakdown`
chart stacked bars
scope Per model: count of sequencing errors vs state fabrications among failed plans.
how to read Tall sequencing segment = grounded model failing on order; tall fabrication segment = invents unsatisfiable states.
data problems Models with almost no failures (or no plans) give tiny/empty bars — proportion is noisy at low counts.
why this form Stacking the two mutually exclusive failure causes shows both the mix and the absolute volume in one bar.

IWSR ≈ FASR	All success on attempt 1 — confirm with declining P(Valid—k).
IWSR near SR	Efficient corrector (attempts 2–3) — verify with rising P(Valid—k).
IWSR near FASR, high RG	Late-retry luck (attempts 4–5) — stochastic; flat P(Valid—k).

ER¿0.70 · PAS¿0.70	Long-horizon sequencing failure — Phase 2 (Valmeekam).
ER¿0.70 · PAS¡0.30	Runs deep but fabricates — world-model incoherence (Vafa).
ER¡0.30 · PAS¡0.30	Fails immediately — Phase 1 fabrication.
ER 0.30–0.70	Mixed — check temporal distance (low = near-miss, high = broken causal model).

Temporal distance

`temporal_distance_by_model`
chart bars
scope Mean gap (steps) between the last true-precondition step and the actual failure step, sequencing errors only.
how to read Larger = the model stays internally consistent for longer before diverging (a' compositional reach' proxy).
data problems Defined only for sequencing errors; models with none get NaN/absent bars — absence is not zero.
why this form A simple per-model bar suffices because temporal distance is already a per-model mean of a single quantity.

7.5.3 Planning efficiency

FASR by model×domain

`fasr_by_model_domain`
chart grouped bars
scope First-attempt success rate per model across domains — success with no repair.
how to read The retry-free skill signal; compare against SR bars to see how much each model leans on retries.
data problems Bounded [0,1]; zero bars common and meaningful (no first-try solves).
why this form Bars per domain make the zero-inflation and the few non-zero domains immediately legible.

SR vs FASR

`sr_vs_fasr`
chart paired bars
scope Overall success rate beside first-attempt success rate, per model.
how to read The vertical gap between the pair is the Retry Gap; a large gap = retry-dependent success.
data problems Both can be 0 for weak models — the pair collapses and the gap is trivially zero.
why this form Pairing the two bars makes the gap a visual quantity instead of a number to subtract mentally.

FASR by difficulty

`fasr_by_difficulty`
chart bars
scope First-attempt success rate aggregated by difficulty tier.
how to read A monotone decline across tiers is the expected difficulty gradient; a cliff signals a capability boundary.
data problems Requires a populated difficulty field; if tiers are unlabelled or unbalanced the bars mislead.
why this form Binning by difficulty is the natural way to expose the McCoy difficulty cliff.

CoT¿0.70 · SR high	Genuine deliberation — strongest result.
CoT¡0.40 · SR high	Fake reasoning — post-hoc CoT (Kambhampati); report as caveat.
CoT¿0.70 · SR low	Faithful but failing — likely long-horizon (if ER also high).
CoT¡0.40 · SR low	Complete grounding failure — return to Level 0 for a masked vocab issue.

IWSR by model×domain

`iwsr_by_model_domain`
chart grouped bars
scope Iteration-weighted success rate (1/iterations on success, else 0) per model×domain.
how to read Discounts late successes; sits between FASR and SR. Closeness to SR = efficient repair.
data problems Bounded [0,1]; like FASR it is zero-inflated for weak models.
why this form Same grid logic as FASR; bars keep the three efficiency metrics visually comparable.

SR · FASR · IWSR triptych

`sr_fasr_iwsr_by_model`
chart grouped bars
scope All three success metrics side by side per model.
how to read Read the descent SR→IWSR→FASR: a steep drop = retry-inflated; a flat triple = genuine first-try skill.
data problems For SR≈0 models all three vanish together, conveying little beyond' failed'.
why this form Co-plotting the three is the cleanest single picture of retry dependence per model.

Retry gap

`retry_gap_by_model`
chart bars
scope SR - FASR per model; the Stechly retry-dependence signal.
how to read Higher = more of the reported success came only after retries.
data problems Zero gap is ambiguous: either pure first-try skill or total failure (SR=FASR=0). Disambiguate with SR.
why this form A single derived bar per model is appropriate; the gap is one number with a clear sign.

FASR vs IWSR scatter

`fasr_iwsr_scatter`
chart bubble scatter
scope FASR (x) vs IWSR (y) with SR encoded as dot size and colour.
how to read On-diagonal = first-try-dominated; below-diagonal with large dot = success built by efficient repair.
data problems Overplotting at the origin when many models score ˜ 0; jitter or label to separate.
why this form Encoding three metrics (x,y,size/colour) in one scatter is denser than three separate bar charts.

P(Valid — reached k)

`p_valid_given_k`
chart line
scope Probability a plan is valid given the model reached attempt k, across k.
how to read Rising curve = Efficient Corrector (later attempts really are better); flat line = Stochastic Searcher (resampling).
data problems Late-k points rest on few surviving instances — the right tail is high-variance and should be read cautiously.
why this form A line over k is the only form that reveals the shape (rising vs flat) the profiles depend on.

7.5.4 CoT alignment

CoT alignment

`cot_alignment_by_model_domain`
chart grouped bars
scope Mean CoTaL (reasoning-vs-plan alignment) per model×domain.
how to read Higher = the reasoning trace structurally matches the emitted plan.
data problems Mostly NaN: most instances have no separate reasoning plan, so bars are absent or rest on tiny samples.
why this form Bars are fine where data exists; the dominant story here is missingness, documented in the CoT view.

Success rate by CoT flag

`cot_success_rate`
chart paired bars
scope Success rate split by whether CoT was present/enabled.
how to read A lift with CoT suggests reasoning helps; no lift suggests post-hoc rationalisation.
data problems Confounded if CoT presence correlates with model family rather than being randomised.
why this form A two-bar split is the minimal correct contrast for a binary flag.

CoT alignment: valid vs invalid

`cot_alignment_validity`
chart box / violin
scope Distribution of CoTaL for valid versus invalid plans.
how to read Higher alignment on valid plans = faithful reasoning; overlap = alignment doesn' t track correctness.
data problems Needs enough valid AND invalid reasoning plans; the bundled run has few of either (18 both-valid, 8 both-invalid).
why this form A distribution contrast is needed because the claim is about separation, not means.

7.5.5 Composite & cross-domain

Success-rate heatmap

`success_rate_heatmap`
chart heatmap
scope Raw SR per model×domain — no normalisation, so cells are comparable across the whole grid.
how to read Empty columns = domains nobody solved (difficulty or infrastructure); empty rows = models that solved nothing.
data problems Genuinely sparse: many cells are 0, which is informative rather than missing.
why this form Unlike the rank heatmap, raw SR is on one common scale, so a single-scale heatmap is valid.

Failure-mode taxonomy

`failure_mode_taxonomy`
chart quadrant bubble
scope Halluc (x) vs PAS (y), bubble size = FASR, colour = SR — four named failure regimes.
how to read Locate each model in a quadrant; same-SR models in different quadrants fail differently. (See Cross-Domain view.)
data problems Bubbles collapse near an axis when a metric is 0/NaN for many models; small samples make positions unstable.
why this form Two orthogonal failure axes plus two encodings packs the entire failure diagnosis into one figure.

Composite Planning Score

`composite_scores`
chart bars + CI
scope PS per model with the 95% bootstrap CI.
how to read Sort by height for the leaderboard; overlapping CIs = the difference is not resolvable.
data problems Overall PS can sit outside its own per-row bootstrap CI for NaN-heavy models (kimi caveat).
why this form Bars with error bars are the standard, honest way to show a ranked scalar with uncertainty.

Within-domain rank heatmap

`domain_ranking_heatmap`
chart heatmap (faceted)
scope Per-domain normalised rank (0=best,1=worst) for every RANK_METRIC.
how to read Compare cells DOWN a column WITHIN one domain block only — never across domain blocks.
data problems Normalisation is per-domain, so identical colours in different blocks are not comparable scores.
why this form Per-domain normalisation onto one colour scale is what lets many domains share a figure.

Rank variance

`rank_variance`
chart bars
scope Mean std of within-domain rank across domains, per model.
how to read Higher = domain specialist (phi_4 ≈2.39); lower = generalist (gpt_oss ≈1.35).
data problems Computed on raw ranks (not normalised) — magnitudes are run-specific; ordering is robust.
why this form A ranked bar is the right form for a single per-model dispersion scalar.

Domain correlation (Spearman)

`domain_correlation`
chart heatmap
scope Model×model Spearman ρ of per-domain success-rate vectors.
how to read $\rho \approx 1$ = redundant probes; low/negative ρ = complementary models worth keeping both.
data problems Needs several domains with variance; with few domains and many SR=0 ties ρ is dominated by ties.
why this form A correlation matrix is the canonical view for pairwise similarity across a model set.

PS stacked by domain

`ps_by_domain_stacked`
chart stacked bars
scope Each model' s PS split into per-domain contributions.
how to read Same total height can be one tall segment (concentrated) or many even ones (broad) — read the segmentation.
data problems Domains with no PS contribution simply don' t appear; ensure the domain set is comparable across models.
why this form Stacking is the natural encoding for' is this total broad or concentrated'— a pie would hide the total.

Metrics summary table

`metrics_summary_table`
chart table figure
scope All key aggregate metrics rendered as a matplotlib table for archiving.
how to read A static snapshot of the overall table; read it as the numeric source of the bar/heatmap figures.

data problems None beyond whatever NaNs the underlying tables carry; it faithfully shows them.
why this form A rendered table travels with the plot bundle so a figure folder is self-documenting.

Capability radar

`radar_chart`
chart radar
scope Polar pentagon of FASR · IWSR · Exec · IHR · PAS per model (base profile, no CoT bonus).
how to read Area = overall strength; shape = capability signature. A collapsed pentagon = No Grounding.
data problems Equal areas can hide very different shapes; many overlaid models become unreadable — facet for publication.
why this form A radar is justified for a small fixed set of bounded axes where the shape itself is the message.

Why the PNGs are not embedded here

The 27 figures are run-specific — they depend on which models and domains were in the bundle and would be stale the moment the suite is re-run. This catalogue documents what each figure *means and how to read it* so it stays valid across runs; regenerate the actual PNGs with the pipeline against your own `results/plots/` directory.

8 Conclusions

DA RIEMPIRE